C#

From Basics to Advanced

The complete guide: concepts, examples, diagrams & tricky points

- How C# & .NET actually run your code
- OOP, Memory, Collections, Generics, LINQ
- Delegates, Events, Async/Await & Multithreading
- Modern C# (up to C# 14) & interview puzzles

Prepared by Kaif Shaikh • Study & Interview Edition

Table of Contents

1. Introduction to C# and .NET 7

1.1 How to Use This Guide 7
1.2 What is C#? 7
1.3 C# vs .NET vs CLR: Clearing the Confusion 8
1.4 How C# Code Actually Runs 9
1.5 Your First Program 10
1.6 Program Structure, Namespaces and using 11
1.7 Comments and XML Documentation 11
1.8 Console Input and Output 12

2. Variables, Data Types and Operators 13

2.1 Variables 13
2.2 Built-in Data Types 14
2.3 Value Types vs Reference Types 15
2.4 Constants: const vs readonly 16
2.5 Type Conversion 16
2.6 Boxing and Unboxing 17
2.7 Nullable Types and Null Operators 18
2.8 Operators 18
2.9 Default Values 19

3. Control Flow: Decisions and Loops 20

3.1 if / else if / else 20
3.2 switch Statement 20
3.3 switch Expression (C# 8+) 21
3.4 Loops 21
3.5 Jump Statements 23
3.6 Pattern Matching Quick Reference 23

4. Methods and Parameters 25

4.1 Defining Methods 25
4.2 Parameter Passing: By Value vs By Reference 25
4.3 Optional and Named Parameters 26
4.4 params - Variable Number of Arguments 27
4.5 Method Overloading (Compile-time Polymorphism) 27
4.6 Recursion 27
4.7 Local Functions and Static Local Functions 28
4.8 Return Values: Tuples and Multiple Results 28

5. Strings and Arrays 29

5.1 String Basics and Immutability ... 29
5.2 Creating Strings ... 29
5.3 Common String Methods ... 30
5.4 StringBuilder ... 31
5.5 Arrays ... 31
5.6 Span<T> and Memory<T> (High-Performance Slicing) ... 32

6. Memory Management and Garbage Collection ... 34

6.1 Stack vs Heap ... 34
6.2 How the Garbage Collector Works ... 35
6.3 IDisposable and the using Statement ... 35
6.4 Weak References ... 36

7. Object-Oriented Programming (OOP) ... 38

7.1 Classes and Objects ... 38
7.2 The Four Pillars of OOP ... 39
7.3 Access Modifiers ... 39
7.4 Constructors ... 40
7.5 Properties ... 41
7.6 static Members and Static Classes ... 42
7.7 Inheritance ... 42
7.8 Polymorphism: virtual, override, new ... 43
7.9 Abstract Classes ... 44
7.10 Interfaces ... 44
7.11 Abstract Class vs Interface ... 45
7.12 System.Object Members ... 45
7.13 Shallow Copy vs Deep Copy ... 46
7.14 Partial Classes and Partial Methods ... 47
7.15 Nested Classes and the this Keyword ... 47
7.16 Operator Overloading and Conversion Operators ... 47
7.17 Indexers ... 48
7.18 SOLID Principles (Quick Guide) ... 48

8. Structs, Records, Enums, Tuples and Anonymous Types ... 49

8.1 Structs ... 49
8.2 Records (C# 9+) ... 50
8.3 Enums ... 51
8.4 Tuples ... 51
8.5 Anonymous Types ... 52
8.6 Deconstruction for Your Own Types ... 52

9. Exception Handling ... 53

9.1 What is an Exception? ... 53
9.2 try, catch, finally ... 54

9.3 throw vs throw ex ... 54
9.4 Tricky Cases with finally ... 55
9.5 Exception Filters (when) ... 55
9.6 Custom Exceptions ... 55
9.7 Common Built-in Exceptions ... 56
9.8 Guard Clauses (modern helpers) ... 56
9.9 Best Practices ... 57
10. Collections (Complete Guide) ... 58
10.1 Overview and Hierarchy ... 58
10.2 List<T> ... 59
10.3 Dictionary<TKey, TValue> ... 61
10.4 HashSet<T> ... 63
10.5 SortedSet<T>, SortedDictionary<K,V>, SortedList<K,V> ... 63
10.6 Queue<T> - FIFO (First In, First Out) ... 64
10.7 Stack<T> - LIFO (Last In, First Out) ... 64
10.8 LinkedList<T> ... 65
10.9 PriorityQueue<TElement, TPriority> (.NET 6) ... 65
10.10 Non-Generic Collections (Legacy) ... 66
10.11 Concurrent (Thread-Safe) Collections ... 66
10.12 Immutable, ReadOnly and Frozen Collections ... 67
10.13 Big-O Complexity Cheat Sheet ... 68
10.14 Which Collection Should I Use? ... 69
10.15 Iterators and yield return ... 69
10.16 Custom Comparers and Equality ... 70
11. Generics ... 71
11.1 Why Generics? ... 71
11.2 Generic Methods ... 71
11.3 Generic Constraints ... 72
11.4 default(T) and Generic Math ... 72
11.5 Covariance and Contravariance ... 73
11.6 Static Fields in Generic Types ... 73
12. Delegates, Lambdas and Events ... 74
12.1 Delegates ... 74
12.2 Multicast Delegates ... 74
12.3 Built-in Delegates: Func, Action, Predicate ... 75
12.4 Anonymous Methods and Lambda Expressions ... 76
12.5 Closures and Captured Variables ... 76
12.6 Events ... 77
12.7 Expression Trees ... 78
13. LINQ (Language Integrated Query) ... 79

13.1 What is LINQ? 79
13.2 Deferred vs Immediate Execution 80
13.3 Filtering and Projection 80
13.4 Element Operators: First, Single, Last, ElementAt 81
13.5 Ordering 81
13.6 Aggregation 82
13.7 Quantifiers 82
13.8 Grouping 82
13.9 Joins 83
13.10 Set Operators 83
13.11 Partitioning, Paging and Others 84
13.12 IEnumerable vs IQueryable 84
13.13 PLINQ (Parallel LINQ) 85

14. Multithreading, Tasks and Async/Await 86

14.1 Key Concepts 86
14.2 Threads 87
14.3 Tasks 87
14.4 async and await 88
14.5 Sequential vs Concurrent Awaits 89
14.6 Async Pitfalls and Tricky Points 89
14.7 Cancellation 90
14.8 Thread Synchronisation 90
14.9 Parallel Class 91
14.10 Async Streams and Channels 92
14.11 Thread-Safe Lazy Initialisation and Singleton 92

15. Advanced C# Features 93

15.1 Extension Methods 93
15.2 Nullable Reference Types (C# 8+) 93
15.3 Attributes 94
15.4 Reflection 94
15.5 typeof vs GetType vs is 95
15.6 dynamic 95
15.7 Unsafe Code and Pointers 95
15.8 File I/O 96
15.9 JSON Serialization (System.Text.Json) 96
15.10 Dependency Injection 96
15.11 Common Design Patterns in C# 97
15.12 Preprocessor Directives and Conditional Compilation 98
15.13 Date, Time and Numeric Formatting 98

16. C# Version History and Modern Features 100

16.1 Feature Timeline 100
16.2 Modern Features in Action 101
16.3 Top-Level Program for a Minimal API (.NET 8+) 102
17. Tricky Interview Questions and Output Puzzles 103
17.1 Output Prediction Puzzles 103
17.2 Frequently Asked Conceptual Questions 107
17.3 Coding Questions to Practise 110
18. Quick Revision Cheat Sheet 112
18.1 Syntax at a Glance 112
18.2 Top 25 Tricky Points (Revise Before Interviews) 112
18.3 Suggested Study Roadmap 113

1. Introduction to C# and .NET

1.1 How to Use This Guide

> **✎ Definition**
>
> A study guide that explains each C# topic in the same order: a simple explanation, a code example, then the tricky points and interview tips that go with it.

This guide takes you from "what is C#?" all the way to advanced topics like async/await, LINQ internals, memory management and modern C# 12-14 features. Every topic follows the same pattern: **simple explanation, then code example, then tricky points**. Coloured boxes help you scan quickly:

> **☼ Simple Analogy**
>
> A real-world comparison that makes the idea stick.

> **⚡ Tricky Point**
>
> Tricky points: behaviour that surprises people and is loved by interviewers.

> **★ Interview Tip**
>
> Interview tip: how to answer the question in one or two crisp lines.

> **⚠ Watch Out**
>
> Watch out: a common bug or bad practice.

1.2 What is C#?

> **✎ Definition**
>
> **C#** is a modern, type-safe, object-oriented programming language from Microsoft. It runs on .NET and is used for web APIs, desktop apps, games (Unity), cloud services and more.

C# (pronounced "C-sharp") is a **modern, object-oriented, type-safe, general-purpose programming language** created by Microsoft (lead designer: Anders Hejlsberg) and first released in 2002 with .NET Framework 1.0. It is now open-source and runs on Windows, Linux and macOS through .NET.

Key characteristics:

- **Strongly & statically typed**: every variable has a type known at compile time, so many bugs are caught before running.
- **Object-oriented**: classes, inheritance, polymorphism, encapsulation, interfaces.
- **Managed**: memory is handled by the Garbage Collector (no manual `free`/`delete`).
- **Multi-paradigm**: also supports functional style (lambdas, LINQ, records, pattern matching).
- **Versatile**: web (ASP.NET Core), desktop (WPF, WinForms, MAUI), cloud, games (Unity), microservices, worker services, mobile.

> ☼ **Simple Analogy**
>
> Think of C# as the *language you speak* and .NET as the *country* where that language is understood. The country provides roads (runtime), libraries (BCL) and services (GC) so you don't have to build them yourself.

1.3 C# vs .NET vs CLR: Clearing the Confusion

> ✎ **Definition**
>
> **C#** is the language, **.NET** is the platform (runtime + base class libraries + tools), and the **CLR** (Common Language Runtime) is the engine inside .NET that loads, JIT-compiles and runs the code.

Term	What it is	Example
C#	The programming language (syntax & rules)	`int x = 5;`
.NET	The platform: runtime + libraries + tools	.NET 8, .NET 9
CLR	Common Language Runtime: the engine that runs your code	JIT, GC
BCL	Base Class Library: ready-made classes	`List<T>`, `File`, `HttpClient`
SDK	Tools to build: compiler, `dotnet` CLI	`dotnet build`
IL / MSIL / CIL	Intermediate Language: CPU-independent bytecode	inside .dll

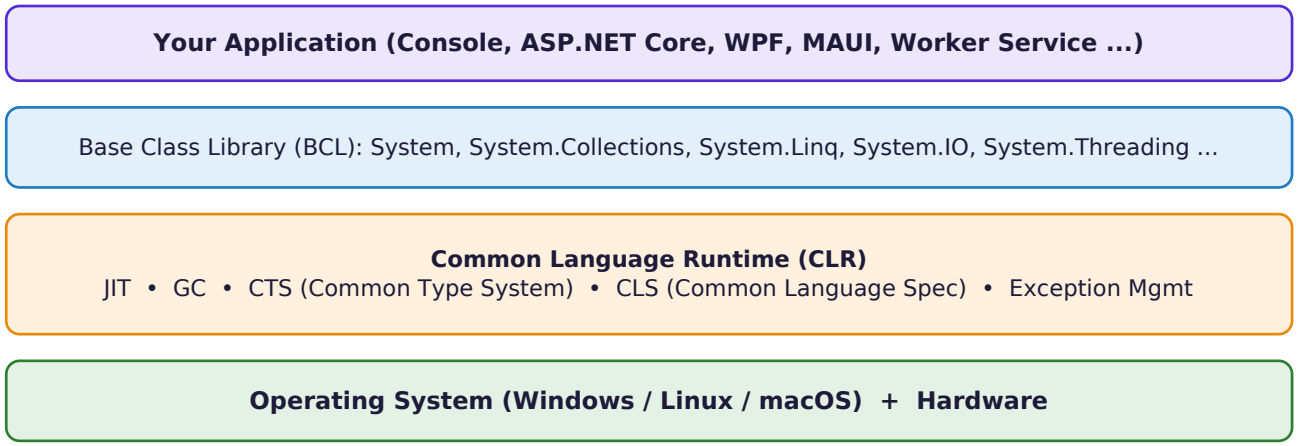


Figure 1.1 - The layers of the .NET platform

.NET Framework vs .NET Core vs .NET 5+

Feature	.NET Framework (1.0 - 4.8.1)	.NET Core (1.0 - 3.1)	.NET 5 / 6 / 7 / 8 / 9 (unified)
Platform	Windows only	Cross-platform	Cross-platform
Open source	Mostly no	Yes	Yes
Status	Maintenance only	Ended	Active (LTS: 6, 8, 10)
Performance	Good	Better	Best
Use today?	Legacy apps	Migrate	Yes, for all new apps

> **⚡ Tricky Point**
>
> There is **no ".NET Core 4"**. Microsoft jumped from .NET Core 3.1 to **.NET 5** to avoid confusion with .NET Framework 4.x and dropped the word "Core" from the runtime name. ASP.NET Core and EF Core still keep the "Core" name.

1.4 How C# Code Actually Runs

> **✎ Definition**
>
> C# source is compiled by Roslyn into **IL** (Intermediate Language) inside an assembly (.dll/.exe). At run time the CLR's **JIT compiler** turns that IL into native machine code for the current CPU.

This is one of the most important concepts. C# is **not** compiled directly to machine code. It happens in two stages:

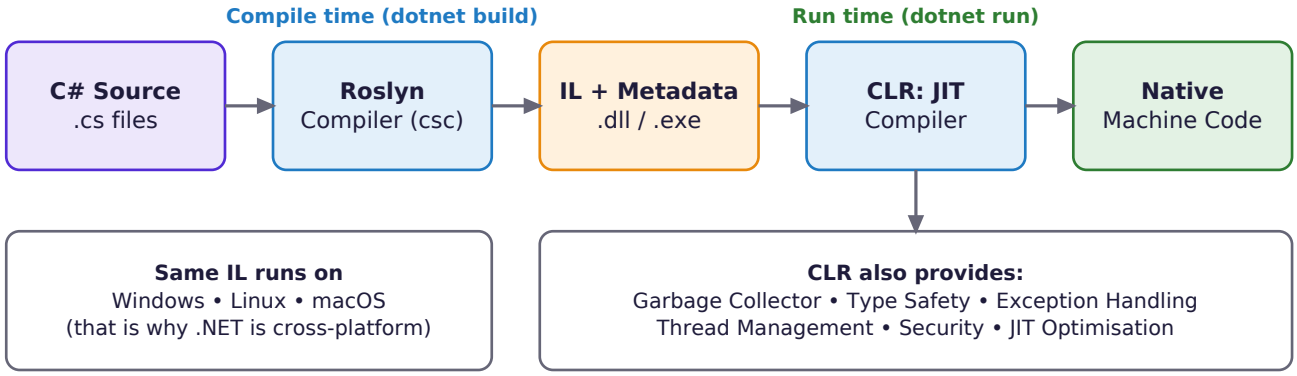


Figure 1.2 - From source code to running machine code

- **Step 1 - Compile time:** The Roslyn compiler (`csc`) converts your `.cs` files into **IL (Intermediate Language)** plus **metadata** (type info), packed into an **assembly** (`.dll` or `.exe`).
- **Step 2 - Run time:** When you run the app, the **CLR** loads the assembly. The **JIT (Just-In-Time) compiler** converts IL into native machine code **method by method, the first time each method is called**, then caches it.
- **Step 3 - Managed execution:** The CLR manages memory (GC), checks types, handles exceptions and threads.

> **☼ Simple Analogy**
>
> IL is like a recipe written in a universal language. The JIT is the local chef who translates it into the local language of the kitchen (x64, ARM) just before cooking.

Important runtime terms

- **Assembly:** compiled unit (.dll/.exe) containing IL, metadata and a manifest (name, version, references).
- **Metadata:** describes every type and member; enables Reflection and IntelliSense.
- **CTS (Common Type System):** defines how types are declared and used, so C#, F# and VB.NET can share types. `int` in C# = `System.Int32` in CTS.
- **CLS (Common Language Specification):** a subset of rules that all .NET languages must follow to interoperate (e.g. don't expose `uint` publicly).
- **Managed code:** code executed by the CLR. **Unmanaged code:** runs directly on the OS (C/C++ DLLs, COM).

- **Tiered compilation:** .NET first JITs quickly (Tier 0) and recompiles hot methods with full optimisation (Tier 1).
- **AOT (Ahead-of-Time):** Native AOT compiles directly to machine code at publish time: faster startup, smaller memory, but no runtime code generation.

> **★ Interview Tip**
>
> "What happens when you run a C# program?" Answer: Roslyn compiles C# to IL in an assembly; at runtime the CLR loads it, the JIT compiles IL to native code per method on first call, and the CLR manages memory with the GC, type safety and exceptions.

1.5 Your First Program

> **✎ Definition**
>
> The smallest complete C# application: code that compiles into an executable and has an entry point (`Main` or top-level statements) where execution starts.

Classic style

```
using System;                // import a namespace

namespace HelloApp           // logical grouping of types
{
    class Program            // every method lives inside a type
    {
        static void Main(string[] args)   // entry point
        {
            Console.WriteLine("Hello, World!");
        }
    }
}
```

Modern style (C# 9+ top-level statements, .NET 6+ template)

```
Console.WriteLine("Hello, World!");
// The compiler generates the Program class and Main method for you.
// 'args' is still available as an implicit variable.
```

Building and running with the CLI

```
dotnet new console -n HelloApp   // create project
cd HelloApp
dotnet build                     // compile to IL (bin/Debug/net8.0/HelloApp.dll)
dotnet run                       // build + run
dotnet publish -c Release        // produce deployable output
```

Valid signatures of Main

```
static void Main()
static void Main(string[] args)
static int Main(string[] args)            // return exit code
static async Task Main(string[] args)     // C# 7.1+
static async Task<int> Main()
```

> **⚡ Tricky Point**
>
> Only **one** entry point is allowed per application. If two classes have `Main`, you get compile error CS0017 unless you specify `<StartupObject>` in the .csproj. With top-level statements, only **one file** in the project may contain them.

1.6 Program Structure, Namespaces and using

> **✎ Definition**
>
> A C# program is organised into **namespaces** (logical groups of types) that contain classes and other types. The `using` directive imports a namespace so its types can be used without their full names.

A namespace prevents name collisions (two `Logger` classes in different libraries). `using` imports a namespace so you don't write the full name.

```
namespace Maximus.Recon.Services;   // file-scoped namespace (C# 10) - less indentation

using System.Text;                  // normal using
using static System.Math;           // use members without class name: Sqrt(16)
using Json = System.Text.Json.JsonSerializer;   // alias
global using System.Linq;           // C# 10: applies to the whole project

public class ReportService
{
    public double Area(double r) => PI * Pow(r, 2);   // thanks to 'using static'
}
```

> **i Note**
>
> With `<ImplicitUsings>enable</ImplicitUsings>` in the .csproj (.NET 6+), common namespaces such as `System`, `System.IO`, `System.Linq`, `System.Collections.Generic` and `System.Threading.Tasks` are imported automatically.

1.7 Comments and XML Documentation

> **✎ Definition**
>
> **Comments** are notes the compiler ignores (`//` and `/* */`). **XML documentation comments** (`///`) describe APIs and feed IntelliSense and generated docs.

```
// single-line comment
/* multi-line
   comment */

/// <summary>
/// Calculates the settlement amount.   (XML doc: shows in IntelliSense)
/// </summary>
/// <param name="amount">Gross amount</param>
/// <returns>Net amount after fees</returns>
public decimal Net(decimal amount) => amount * 0.98m;
```

1.8 Console Input and Output

✎ Definition

Reading input from and writing output to the terminal through the `System.Console` class (`ReadLine`, `Write`, `WriteLine`).

```
Console.Write("Enter your name: ");
string? name = Console.ReadLine();          // reads a line (may be null)
Console.WriteLine($"Hello {name}!");         // string interpolation

Console.Write("Age: ");
int age = int.Parse(Console.ReadLine()!);    // throws if not a number
if (int.TryParse(Console.ReadLine(), out int safeAge))   // safe way
    Console.WriteLine($"Next year you'll be {safeAge + 1}");
```

2. Variables, Data Types and Operators

2.1 Variables

> **✎ Definition**
>
> A **variable** is a named storage location that holds a value of a specific type. C# is statically typed, so the type is fixed at compile time (explicitly, or inferred with `var`).

A variable is a **named storage location** with a fixed type. Syntax: `type name = value;`

```
int count = 10;
string city = "Pune";
double price = 99.5;
bool isActive = true;
char grade = 'A';
var total = 250m;          // type inferred as decimal at COMPILE time
```

Naming rules: start with a letter or `_`, no spaces, case-sensitive (`age` and `Age` are different), cannot be a keyword (use `@class` if you really must). Conventions: **camelCase** for locals and parameters, **PascalCase** for methods, properties, classes, and `_camelCase` for private fields.

var - implicit typing

- `var` is **not** dynamic. The type is fixed at compile time from the right-hand side.
- Must be initialised on the same line: `var x;` is an error. `var x = null;` is also an error (no type).
- Only for local variables, not fields or parameters.

```
var list = new List<string>();   // list is List<string>
list = 5;                         // compile error - type cannot change
```

> **⚡ Tricky Point**
>
> `var` vs `dynamic` vs `object`: `var` is resolved at compile time; `object` is the base type (needs casting, boxes value types); `dynamic` skips compile-time checking and resolves members at **runtime** (errors appear as `RuntimeBinderException`).

Target-typed new (C# 9)

```
List<int> numbers = new();          // type taken from the left side
Dictionary<string, int> map = new() { ["a"] = 1 };
```

2.2 Built-in Data Types

> ✎ **Definition**
>
> The predefined types C# provides as keywords (`int`, `double`, `decimal`, `bool`, `char`, `string`, `object`…). Each keyword is an alias for a .NET type, e.g. `int` = `System.Int32`.

C# types are split into **value types** (hold data directly) and **reference types** (hold a reference/address to data on the heap).

C# keyword	.NET type	Size	Range / Notes
`byte`	System.Byte	1 byte	0 to 255
`sbyte`	System.SByte	1 byte	-128 to 127
`short`	System.Int16	2 bytes	-32,768 to 32,767
`ushort`	System.UInt16	2 bytes	0 to 65,535
`int`	System.Int32	4 bytes	about -2.1 billion to 2.1 billion

C# keyword	.NET type	Size	Range / Notes
`uint`	System.UInt32	4 bytes	0 to about 4.29 billion
`long`	System.Int64	8 bytes	about +/- 9.2 x 10^18
`float`	System.Single	4 bytes	~6-9 digits precision, suffix `f`
`double`	System.Double	8 bytes	~15-17 digits precision (default for decimals)
`decimal`	System.Decimal	16 bytes	28-29 digits, base-10, suffix `m` - use for money
`char`	System.Char	2 bytes	One UTF-16 character, single quotes
`bool`	System.Boolean	1 byte	true / false (not 0/1)
`string`	System.String	reference	Immutable sequence of chars
`object`	System.Object	reference	Base of every type
`nint`/`nuint`	IntPtr/UIntPtr	4 or 8	Native-sized integers (C# 9)

> ⚡ **Tricky Point**
>
> `0.1 + 0.2 == 0.3` is **false** with `double` (result is 0.30000000000000004) because double is binary floating point. With `decimal` (`0.1m + 0.2m == 0.3m`) it is **true**. Always use `decimal` for currency and settlement amounts.

```
double d = 0.1 + 0.2;
Console.WriteLine(d == 0.3);          // False
Console.WriteLine(0.1m + 0.2m == 0.3m); // True
Console.WriteLine(int.MaxValue + 1);  // -2147483648 (silent overflow!)
checked { int x = int.MaxValue; x++; } // throws OverflowException
Console.WriteLine(10 / 3);            // 3   (integer division)
Console.WriteLine(10 / 3.0);          // 3.3333333333333335
Console.WriteLine(1.0 / 0);           // Infinity (no exception for double)
// Console.WriteLine(1 / 0);          // compile error; with variables -> DivideByZeroException
```

Literals and digit separators

```
int million = 1_000_000;      // underscore for readability (C# 7)
int hex = 0xFF;               // 255
int bin = 0b1010_1010;        // 170
long big = 5_000_000_000L;
float f = 3.14f; decimal m = 19.99m; double d = 1e-3;
```

2.3 Value Types vs Reference Types

✎ Definition

A **value type** variable holds its data directly, so assignment copies the data. A **reference type** variable holds a reference to an object on the heap, so assignment copies only the reference.

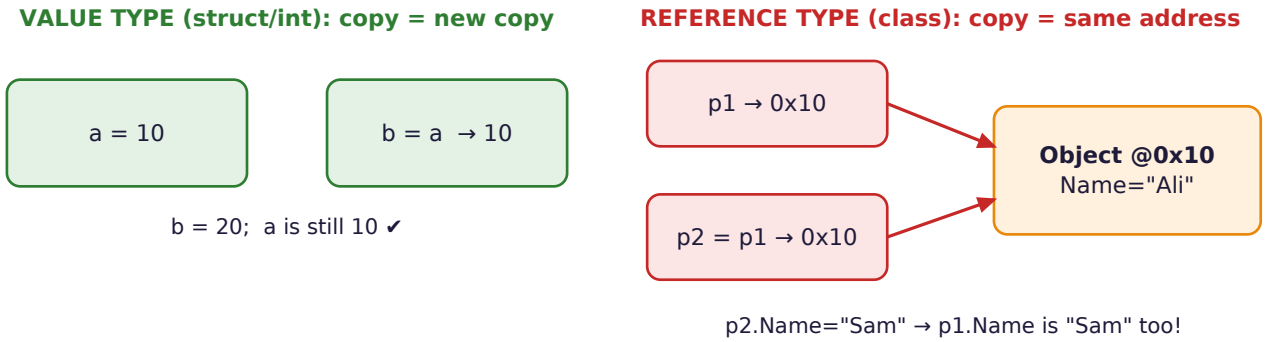


Figure 2.1 - Copying a value type vs copying a reference type

Value types	Reference types
int, double, bool, char, decimal, enum, **struct**, tuples (ValueTuple)	**class**, string, array, interface, delegate, record (class), object
Variable contains the data itself	Variable contains a reference (address) to an object
Assignment copies the whole value	Assignment copies only the reference
Cannot be null (unless `int?`)	Can be null
Usually on the stack (or inline inside the containing object)	Object always lives on the managed heap
Derive from System.ValueType	Derive from System.Object

⚡ Tricky Point

"Value types live on the stack" is only half-true. A value type that is a **field of a class** lives on the heap inside that object. A local `int` captured by a lambda is also moved to the heap. The correct statement: value types are stored **where they are declared**; reference-type objects always go on the heap.

2.4 Constants: const vs readonly

> ✎ **Definition**
>
> `const` is a compile-time constant whose value is baked into the code. `readonly` is a field that can be assigned only at declaration or in a constructor, so its value is fixed at run time.

```
public class Config
{
    public const double Pi = 3.14159;           // compile-time constant, implicitly static
    public readonly DateTime CreatedAt;          // set at declaration or in constructor
    public static readonly HttpClient Client = new();   // runtime constant, one instance

    public Config() { CreatedAt = DateTime.Now; }   // allowed
}
```

const	readonly
Value fixed at compile time	Value fixed at runtime (once)
Only primitives, string, enum, null	Any type
Implicitly static	Instance or static
Value is **copied into calling assemblies**	Read from the field at runtime

> ⚡ **Tricky Point**
>
> Versioning trap: if Library A has `public const int Max = 10;` and App B uses it, the value 10 is **baked into B's IL**. Change A to 20 and redeploy only A - B still uses 10 until it is recompiled. Use `static readonly` for values that may change.

2.5 Type Conversion

> ✎ **Definition**
>
> **Type conversion** changes a value from one type to another: **implicit** (safe, automatic), **explicit** (a cast that may lose data), or through helpers such as `Convert`, `Parse` and `TryParse`.

Implicit (safe, no data loss)

```
int i = 100; long l = i; double d = i;   // smaller -> bigger: automatic
```

Explicit casting (may lose data)

```
double price = 99.99;
int whole = (int)price;           // 99 (truncates, does NOT round)
int big = 300; byte b = (byte)big; // 44 (300 - 256): overflow wraps silently
```

Convert, Parse and TryParse

```
int a = int.Parse("123");              // throws FormatException on "abc", ArgumentNullException on null
int b = Convert.ToInt32("123");        // returns 0 for null, throws on "abc"
bool ok = int.TryParse("abc", out int c); // ok=false, c=0, NO exception - best for user input
int r = Convert.ToInt32(2.5);   // 2  (banker's rounding: to nearest even)
int s = Convert.ToInt32(3.5);   // 4
int t = (int)2.9;               // 2  (cast truncates)
```

> **⚡ Tricky Point**
>
> `Convert.ToInt32(null)` returns **0** but `int.Parse(null)` throws `ArgumentNullException`. `Math.Round(2.5)` returns **2**, not 3 (MidpointRounding.ToEven). Use `Math.Round(2.5, MidpointRounding.AwayFromZero)` to get 3.

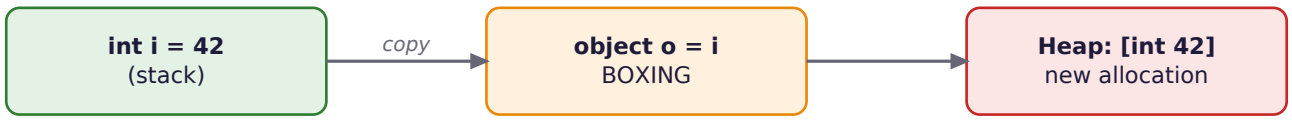
as and is operators

```
object o = "hello";
string? s = o as string;      // returns null if conversion fails (no exception)
if (o is string str)          // type pattern: test + cast in one step
    Console.WriteLine(str.Length);
// 'as' works only with reference types and nullable types: (o as int) is an error, use (o as int?)
```

2.6 Boxing and Unboxing

> **✎ Definition**
>
> **Boxing** wraps a value type in an object on the heap so it can be treated as `object`. **Unboxing** extracts the value back with an explicit cast. Both cost time and memory.

Boxing = converting a value type to `object` (or an interface). The CLR allocates a new object on the heap and copies the value. **Unboxing** = extracting it back with an explicit cast.



int j = (int)o; ← UNBOXING (copy back + type check, InvalidCastException if wrong type)

Figure 2.2 - Boxing allocates on the heap

```
int i = 42;
object o = i;          // boxing
int j = (int)o;        // unboxing
long k = (long)o;      // InvalidCastException! must unbox to the EXACT type
long k2 = (long)(int)o;   // correct: unbox to int, then convert

ArrayList list = new ArrayList();
list.Add(1);           // boxing on every Add - why generic List<int> is faster
```

> **⚠ Watch Out**
>
> Boxing in hot loops (old `ArrayList`, `string.Format` with value types, calling interface methods on structs) creates garbage and hurts performance. Generics remove most boxing.

2.7 Nullable Types and Null Operators

> ✎ **Definition**
>
> A **nullable value type** (`int?` = `Nullable<int>`) can hold a value or `null`. Null operators (`?.`, `??`, `??=`, `!`) work safely with values that may be null.

```
int? score = null;               // Nullable<int>
if (score.HasValue) Console.WriteLine(score.Value);
int final = score ?? 0;          // null-coalescing: use 0 if null
int val = score.GetValueOrDefault();   // 0

string? name = null;
int? len = name?.Length;         // null-conditional: null instead of NullReferenceException
string upper = name?.ToUpper() ?? "UNKNOWN";
name ??= "Guest";                // assign only if null (C# 8)
var first = customers?[0]?.Address?.City;   // safe chain
```

> ⚡ **Tricky Point**
>
> `int? a = null; int? b = 5; a + b` gives **null** (lifted operators). `a > b` and `a < b` are both **false**. But `a == null` is true.

2.8 Operators

> ✎ **Definition**
>
> **Operators** are symbols that perform operations on operands: arithmetic, comparison, logical, bitwise, assignment, conditional (`?:`) and type operators (`is`, `as`, `typeof`).

Category	Operators
Arithmetic	`+ - * / % ++ --`
Comparison	`== != > < >= <=`
Logical	`&& || !` (short-circuit) and `& | ^` (always evaluate both)
Bitwise	`& | ^ ~ << >> >>>` (>>> unsigned shift, C# 11)
Assignment	`= += -= *= /= %= ??=`
Null	`?. ?[] ?? ??= !` (null-forgiving)
Type	`is as typeof sizeof nameof default`
Other	`?:` ternary, `=>` lambda, `..` range, `^` index-from-end

Prefix vs postfix increment

```
int x = 5;
int a = x++;   // a = 5, x = 6   (use, then increment)
int b = ++x;   // b = 7, x = 7   (increment, then use)
int i = 1;
i = i++ + ++i; // 1 + 3 = 4  (classic interview puzzle)
```

Short-circuit evaluation

```
if (user != null && user.IsActive)  // safe: right side runs only if left is true
if (user != null & user.IsActive)   // NullReferenceException if user is null!
```

Ternary and nameof

```
string status = age >= 18 ? "Adult" : "Minor";
Console.WriteLine(nameof(status));   // "status" - refactor-safe string
throw new ArgumentNullException(nameof(customerId));
```

⚡ Tricky Point

Operator precedence: `*` and `/` before `+`; `&&` before `||`; `??` has very low precedence. `a ?? b + c` means `a ?? (b + c)`. When in doubt, use parentheses.

2.9 Default Values

✎ Definition

The value a variable or field gets when it is not explicitly assigned: `0` for numbers, `false` for bool, `null` for reference types. Available in code as `default(T)`.

Type	default(T)
Numeric types	0
bool	false
char	'\0'
Reference types (class, string, array)	null
struct	all fields set to their defaults
Nullable<T>	null

Fields of a class are automatically set to defaults. **Local variables are not** - using an unassigned local is a compile error (CS0165).

3. Control Flow: Decisions and Loops

3.1 if / else if / else

✎ Definition

A **conditional statement** that runs a block of code only when a boolean condition is true, with optional `else if` and `else` branches for the other cases.

```
decimal amount = 15000m;
if (amount > 100000) Console.WriteLine("High value - needs approval");
else if (amount > 10000) Console.WriteLine("Medium value");
else Console.WriteLine("Low value");
```

⚡ Tricky Point

The condition must be a `bool`. `if (count)` is a compile error in C# (unlike C/JavaScript). Also `if (x = 5)` does not compile, preventing the classic "assignment instead of comparison" bug.

3.2 switch Statement

✎ Definition

A **multi-way branch** that compares one value against several case labels and runs the matching section. C# does not allow implicit fall-through between non-empty cases.

```
switch (day)
{
    case "Sat":
    case "Sun":                         // stacking cases = OR
        Console.WriteLine("Weekend");
        break;
    case "Fri":
        Console.WriteLine("Almost weekend");
        break;
    default:
        Console.WriteLine("Weekday");
        break;
}
```

⚡ Tricky Point

C# does **not** allow implicit fall-through. Every non-empty case must end with `break`, `return`, `throw`, `continue` or `goto case "X";`. Empty cases stacked together are allowed.

switch with patterns and when guards (C# 7+)

```
switch (shape)
{
    case Circle c when c.Radius > 10: Console.WriteLine("Big circle"); break;
    case Circle c:                    Console.WriteLine($"Circle r={c.Radius}"); break;
    case Rectangle { Width: var w, Height: var h } when w == h:
                                      Console.WriteLine("Square"); break;
    case null:                        Console.WriteLine("null shape"); break;
    default:                          Console.WriteLine("Unknown"); break;
}
```

Order matters: the compiler errors if a case is unreachable because an earlier case already covers it.

3.3 switch Expression (C# 8+)

> **✎ Definition**
>
> A compact, **expression-based** form of switch that *returns a value*, using `pattern => result` arms and _ as the catch-all (discard) arm.

A concise form that **returns a value**.

```
string GetFee(string cardType) => cardType switch
{
    "VISA"   => "1.8%",
    "MASTER" => "1.9%",
    "RUPAY"  => "0%",
    _        => throw new ArgumentException("Unknown card")   // _ = discard (default)
};

decimal Discount(Customer c) => c switch
{
    { IsPremium: true, Years: > 5 } => 0.20m,     // property pattern + relational
    { IsPremium: true }             => 0.10m,
    { Years: >= 2 and < 5 }         => 0.05m,     // 'and' pattern (C# 9)
    _                               => 0m
};
```

> **⚠ Watch Out**
>
> If no arm matches and there is no `_`, a `SwitchExpressionException` is thrown at runtime (the compiler only warns).

3.4 Loops

> **✎ Definition**
>
> **Loops** repeat a block of code: `for` (counter-based), `while` (checks before each pass), `do-while` (runs at least once) and `foreach` (walks over a collection).

for loop - when you know the count

```
for (int i = 0; i < 5; i++)
    Console.Write(i + " ");          // 0 1 2 3 4
for (int i = 10, j = 0; i > j; i--, j++) { }   // multiple variables
for (;;) { break; }                   // infinite loop
```

while - check first, may run zero times

```
int retries = 0;
while (retries < 3 && !Connect()) retries++;
```

do-while - runs at least once

```
string input;
do { input = Console.ReadLine() ?? ""; } while (input != "exit");
```

foreach - iterate any IEnumerable

```
var cities = new List<string> { "Mumbai", "Pune", "Delhi" };
foreach (var city in cities) Console.WriteLine(city);
foreach (var (key, value) in dict) Console.WriteLine($"{key}={value}");  // deconstruction
```

> ⚡ **Tricky Point**
>
> You **cannot modify** a `List<T>` (Add/Remove) inside a `foreach` over it - you get `InvalidOperationException: Collection was modified`. Solutions: iterate a copy (`list.ToList()`), use a reverse `for` loop, or use `list.RemoveAll(predicate)`.

```
for (int i = list.Count - 1; i >= 0; i--)    // reverse loop is safe for removal
    if (list[i] < 0) list.RemoveAt(i);
list.RemoveAll(x => x < 0);                    // best option
```

> ⚡ **Tricky Point**
>
> The `foreach` iteration variable is read-only: `foreach (var x in arr) x = 5;` does not compile. For a `List<MyStruct>` you cannot change `item.Field` either, because you get a copy.

How foreach works internally

`foreach` is syntactic sugar. The compiler turns it into roughly this (it needs only a `GetEnumerator()` method - duck typing):

```
using (var e = cities.GetEnumerator())
{
    while (e.MoveNext())
    {
        var city = e.Current;
        Console.WriteLine(city);
    }
}
```

3.5 Jump Statements

✎ Definition

Statements that move control elsewhere: `break` (leave a loop or switch), `continue` (skip to the next iteration), `return` (leave a method) and `goto` (jump to a label).

Statement	Effect
`break`	Exit the nearest loop or switch
`continue`	Skip to the next iteration
`return`	Exit the method (optionally with a value)
`goto`	Jump to a label or `case` (rarely used)
`throw`	Raise an exception
`yield return` / `yield break`	Produce / stop producing items in an iterator

```
for (int i = 0; i < 10; i++)
{
    if (i % 2 == 0) continue;   // skip even
    if (i > 7) break;           // stop at 9
    Console.Write(i);           // 1 3 5 7
}
```

⚡ Tricky Point

`break` only exits the **innermost** loop. To break out of nested loops, use a flag, put the loops in a method and `return`, or (rarely) `goto`.

3.6 Pattern Matching Quick Reference

✎ Definition

Pattern matching tests a value's shape and extracts data in one step, using type, constant, relational, logical, property, positional and list patterns with `is` and `switch`.

Pattern	Example	Version
Type pattern	`if (o is int n)`	C# 7
Constant	`x is null`, `x is 5`	C# 7
Negation	`x is not null`	C# 9
Relational	`age is >= 18 and < 60`	C# 9
Logical	`c is 'a' or 'e' or 'i'`	C# 9
Property	`p is { Age: > 18, Name: "Ali" }`	C# 8
Extended property	`o is { Address.City: "Pune" }`	C# 10
Positional	`point is (0, 0)`	C# 8
var	`is var x` (always matches)	C# 7
Discard	`_`	C# 8
List pattern	`arr is [1, 2, ..]`, `[_, .., var last]`	C# 11

```
int[] nums = { 1, 2, 3, 4 };
if (nums is [1, .., var last]) Console.WriteLine(last);   // 4
string Describe(int[] a) => a switch
{
    [] => "empty",
    [var x] => $"one: {x}",
    [var x, var y] => $"two: {x},{y}",
    [var first, ..] => $"starts with {first}"
};
```

★ Interview Tip

Prefer `is null` / `is not null` over `== null`. A class can overload `==`, so `== null` might run custom code; `is null` always checks the actual reference.

4. Methods and Parameters

4.1 Defining Methods

> ✎ **Definition**
>
> A **method** is a named block of code that performs a task. It has an access modifier, return type, name and parameter list, and is called to reuse logic.

```
//  access   modifier  return  name     parameters
    public   static    int     Add(int a, int b)
    {
        return a + b;
    }

public int Square(int x) => x * x;          // expression-bodied member
public void Log(string msg) => Console.WriteLine(msg);
```

A method's **signature** is its name plus parameter types/kinds (`ref`/`out`/`in`). The return type is **not** part of the signature for overloading.

4.2 Parameter Passing: By Value vs By Reference

> ✎ **Definition**
>
> **By value** (the default) passes a copy of the argument. **By reference** (`ref`, `out`, `in`) passes the variable itself, so the method can read or change the caller's variable.

By default, C# passes **a copy of the variable**. For a value type, it copies the data. For a reference type, it copies the **reference** - so the method can modify the object but cannot make the caller's variable point to a new object.

```
void ChangeValue(int x) { x = 100; }
void ChangeObject(Person p) { p.Name = "Changed"; }   // affects the caller's object
void Reassign(Person p) { p = new Person { Name = "New" }; }  // caller NOT affected

int n = 5; ChangeValue(n);           // n is still 5
var per = new Person { Name = "Ali" };
ChangeObject(per);                   // per.Name == "Changed"
Reassign(per);                       // per.Name still "Changed"
```

> ⚡ **Tricky Point**
>
> The classic confusion: "Reference types are passed by reference" is **wrong**. Everything is passed by value by default; for reference types the *value* being copied is the reference. Only `ref`/`out`/`in` pass the variable itself.

ref, out, in

```
void Swap(ref int a, ref int b) { (a, b) = (b, a); }
int x = 1, y = 2; Swap(ref x, ref y);   // x=2, y=1. Must be initialised before passing

bool TryDivide(int a, int b, out int result)
{
    if (b == 0) { result = 0; return false; }   // out MUST be assigned before return
    result = a / b; return true;
}
if (TryDivide(10, 2, out var r)) Console.WriteLine(r);   // inline out declaration
TryDivide(1, 0, out _);                                   // discard
void Print(in LargeStruct s) { /* s is read-only; passed by reference for speed */ }

void Reset(ref Person p) { p = new Person(); }   // NOW the caller's variable changes
```

Modifier	Must initialise before call?	Must assign inside method?	Can modify inside?
`ref`	Yes	No	Yes

Modifier	Must initialise before call?	Must assign inside method?	Can modify inside?
`out`	No	Yes	Yes
`in`	Yes	No	No (read-only)
`ref readonly` (C# 12)	Yes	No	No

⚡ Tricky Point

You cannot overload methods differing **only** by `ref` vs `out` (`void M(ref int x)` and `void M(out int x)` conflict). But `M(int x)` and `M(ref int x)` can coexist. `async` methods and iterators cannot have `ref`/`out`/`in` parameters.

4.3 Optional and Named Parameters

✎ Definition

Optional parameters have default values, so callers may omit them. **Named arguments** pass values by parameter name (`age: 30`) instead of position.

```
void SendMail(string to, string subject = "No subject", bool isHtml = false) { }

SendMail("a@b.com");
SendMail("a@b.com", isHtml: true);                // skip subject using named arg
SendMail(subject: "Hi", to: "a@b.com");           // any order
```

⚡ Tricky Point

Default values must be compile-time constants and are **baked into the caller** (like `const`). Changing the default in a library requires recompiling callers. Optional parameters must come after required ones.

4.4 params - Variable Number of Arguments

✎ Definition

The `params` modifier lets a method accept a **variable number of arguments**, which the compiler packs into an array (or, from C# 13, another collection type).

```
int Sum(params int[] numbers) => numbers.Sum();
Sum();            // 0
Sum(1, 2, 3);     // 6
Sum(new[] {4, 5}); // 9
// C# 13: params works with collections too: void Log(params ReadOnlySpan<string> items)
```

Only one `params` parameter is allowed and it must be last.

4.5 Method Overloading (Compile-time Polymorphism)

✎ Definition

Method overloading means defining several methods with the same name but different parameter lists. The compiler chooses which one to call at compile time.

Same name, different parameter list.

```
public int Area(int side) => side * side;
public int Area(int l, int w) => l * w;
public double Area(double r) => Math.PI * r * r;
// public double Area(int side)  -> ERROR: differs only by return type
```

⚡ Tricky Point

Overload resolution picks the **most specific** match. `Print(5)` with `Print(long)` and `Print(double)` overloads picks `long` (int converts to both, but long is "better"). `Print(null)` with `Print(string)` and `Print(object)` picks `string`. With `Print(string)` and `Print(StringBuilder)`, `Print(null)` is **ambiguous** (compile error).

4.6 Recursion

✎ Definition

Recursion is when a method calls itself to solve a smaller instance of the same problem. It needs a base case that stops the calls.

```
long Factorial(int n) => n <= 1 ? 1 : n * Factorial(n - 1);
int Fib(int n) => n < 2 ? n : Fib(n - 1) + Fib(n - 2);   // O(2^n) - slow!
```

⚠ Watch Out

Each recursive call uses stack space. Deep recursion (around 10,000+ frames) causes `StackOverflowException`, which **cannot be caught** and kills the process. Use iteration or memoization for large inputs.

4.7 Local Functions and Static Local Functions

> **✎ Definition**
>
> A **local function** is a method declared inside another method and visible only there. A `static` local function cannot capture the enclosing method's variables.

```
public int Process(int[] data)
{
    return data.Where(IsValid).Sum();

    bool IsValid(int x) => x > 0 && x < limit;   // can use locals declared before calls
    // static bool IsValid2(int x) => x > 0;      // 'static' = cannot capture variables
}
```

Local functions are better than lambdas for helpers: no delegate allocation, can be recursive, support `ref`/`out`, and can be iterators.

4.8 Return Values: Tuples and Multiple Results

> **✎ Definition**
>
> Ways to return more than one value from a method: **tuples** `(int, string)`, out parameters, or a custom class, record or struct.

```
(int min, int max) GetRange(int[] arr) => (arr.Min(), arr.Max());
var (lo, hi) = GetRange(new[] { 3, 9, 1 });   // deconstruction: lo=1, hi=9
var r = GetRange(new[] { 3, 9, 1 });
Console.WriteLine(r.max);
```

ref returns

```
ref int FindRef(int[] arr, int value)
{
    for (int i = 0; i < arr.Length; i++)
        if (arr[i] == value) return ref arr[i];
    throw new InvalidOperationException();
}
int[] a = { 1, 2, 3 };
ref int slot = ref FindRef(a, 2);
slot = 99;            // a is now { 1, 99, 3 }
```

5. Strings and Arrays

5.1 String Basics and Immutability

> ✎ **Definition**
>
> A `string` is an **immutable** sequence of UTF-16 characters (a reference type). Any "change" creates a new string object; the original never changes.

`string` (alias of `System.String`) is a **reference type** that is **immutable**: every "modification" creates a brand-new string object. It *behaves* like a value type for equality because `==` is overloaded to compare content.

```
string s = "Hello";
s.ToUpper();               // returns new string, s is unchanged!
s = s.ToUpper();           // correct
string a = "hi", b = "hi";
Console.WriteLine(a == b);                    // True  (content)
Console.WriteLine(object.ReferenceEquals(a, b)); // True  (literals are interned)
string c = new string(new[] {'h','i'});
Console.WriteLine(object.ReferenceEquals(a, c)); // False (different object, same content)
```

> ☼ **Simple Analogy**
>
> A string is like a printed page. You cannot erase a word; you must print a new page. StringBuilder is like a whiteboard you can keep editing.

> ⚡ **Tricky Point**
>
> **String interning**: the CLR keeps one copy of each literal in an "intern pool", so identical literals share the same object. Strings built at runtime are not interned unless you call `string.Intern(s)`.

5.2 Creating Strings

> ✎ **Definition**
>
> Ways to build string values: literals, **verbatim** strings (`@"..."`), **interpolated** strings (`$"..."`), **raw** string literals (`"""..."""`), concatenation and `string.Join`/`Format`.

```
string name = "Kaif";
string path1 = "C:\\data\\file.txt";             // escape sequences
string path2 = @"C:\data\file.txt";              // verbatim: backslash literal, multi-line allowed
string msg = $"Hi {name}, total = {amount:C2}";  // interpolation with format
string padded = $"{id,-10}|{amt,12:N2}";          // alignment: -10 left, 12 right
string json = """
    { "name": "Ali", "age": 30 }
    """;                                          // raw string literal (C# 11): no escaping
string rawInterp = $$"""{ "name": "{{name}}" }""";  // $$ means {{ }} for holes
string empty = string.Empty;                     // same as ""
```

Escape sequences: `\n` newline, `\t` tab, `\\` backslash, `\"` quote, `\0` null, `\u0041` unicode.

5.3 Common String Methods

✎ Definition

The built-in methods of the `String` class for searching, comparing, slicing, splitting, trimming, padding and changing case. All of them return new strings.

Method	Example	Result
Length	`"hello".Length`	5
ToUpper/ToLower	`"Hi".ToUpper()`	"HI"
Trim/TrimStart/TrimEnd	`" a ".Trim()`	"a"
Substring(start, len)	`"abcdef".Substring(2, 3)`	"cde"
IndexOf / LastIndexOf	`"banana".IndexOf('n')`	2 (-1 if absent)
Contains/StartsWith/EndsWith	`"abc".Contains("b")`	true
Replace	`"a-b".Replace("-", "+")`	"a+b"
Split	`"a,b,c".Split(',')`	["a","b","c"]
Join	`string.Join("-", arr)`	"a-b-c"
Concat	`string.Concat("a","b")`	"ab"
PadLeft/PadRight	`"7".PadLeft(3,'0')`	"007"
IsNullOrEmpty / IsNullOrWhiteSpace	`string.IsNullOrWhiteSpace(" ")`	true
Equals with comparison	`a.Equals(b, StringComparison.OrdinalIgnoreCase)`	case-insensitive
Compare	`string.Compare("a","b")`	negative
ToCharArray / indexer	`"abc"[1]`	'b'
Insert/Remove	`"abc".Remove(1,1)`	"ac"
Format	`string.Format("{0:N2}", 1234.5)`	"1,234.50"

```
string csv = "TXN001, 500.00 , SUCCESS";
string[] parts = csv.Split(',', StringSplitOptions.TrimEntries | StringSplitOptions.RemoveEmptyEntries);
bool same = "VISA".Equals("visa", StringComparison.OrdinalIgnoreCase);   // true
string reversed = new string("hello".Reverse().ToArray());              // "olleh"
```

⚡ Tricky Point

Use `StringComparison.Ordinal` / `OrdinalIgnoreCase` for keys, codes, file paths. Culture-aware comparison (`ToLower()`, default `Compare`) can give surprising results, e.g. the Turkish "I" problem where `"FILE".ToLower()` does not equal `"file"` under the tr-TR culture.

5.4 StringBuilder

> ✎ **Definition**
>
> `StringBuilder` is a **mutable** character buffer for building a string step by step (for example in loops) without creating a new string on every change.

```
var sb = new StringBuilder();
for (int i = 0; i < 1000; i++)
    sb.Append(i).Append(',');      // fluent: Append returns the same builder
sb.Length--;                        // remove last comma
sb.Insert(0, "[").Append(']');
sb.AppendLine().AppendFormat("Total {0}", 1000);
string result = sb.ToString();
```

string concatenation `+=`	StringBuilder
New string object every time: O(n^2) copying in a loop	Internal resizable buffer: amortised O(n)
Fine for few (< ~5) concatenations	Best for loops / many appends

string concatenation `+=`	StringBuilder
Immutable, thread-safe to read	Mutable, not thread-safe

> ⚡ **Tricky Point**
>
> `string s = "a" + "b" + "c";` with literals is combined at **compile time** into "abc". `s1 + s2 + s3` (variables) compiles to one `string.Concat(s1, s2, s3)` call - so StringBuilder only matters for **loops** or many separate statements.

5.5 Arrays

> ✎ **Definition**
>
> An **array** is a fixed-size, zero-indexed collection of elements of one type, stored contiguously. C# has single-dimensional, multi-dimensional `[,]` and jagged `[][]` arrays.

An array is a **fixed-size**, zero-indexed collection of elements of the same type. Arrays are reference types (derived from `System.Array`), even when the elements are value types.

```
int[] nums = new int[5];                // {0,0,0,0,0}
int[] primes = { 2, 3, 5, 7 };
string[] names = new string[] { "A", "B" };
int[] modern = [1, 2, 3];               // collection expression (C# 12)
Console.WriteLine(primes.Length);       // 4
Console.WriteLine(primes[^1]);          // 7  (index from end, C# 8)
int[] mid = primes[1..3];               // {3, 5}  (range: start inclusive, end exclusive)
```

Multi-dimensional vs jagged arrays

```
int[,] grid = new int[3, 4];            // rectangular 3x4, one block of memory
grid[0, 1] = 5;
int rows = grid.GetLength(0);           // 3
int cols = grid.GetLength(1);           // 4

int[][] jagged = new int[3][];          // array of arrays - rows can differ in length
jagged[0] = new[] { 1, 2 };
jagged[1] = new[] { 3, 4, 5, 6 };
jagged[2] = new int[0];
Console.WriteLine(jagged[1][2]);        // 5
```

Rectangular `int[,]`	Jagged `int[][]`
All rows same length	Rows can have different lengths
Single object, accessed `[i, j]`	Array of separate array objects, `[i][j]`
Slightly slower element access	Usually faster (JIT optimises 1-D arrays)

Useful Array methods

```
int[] arr = { 5, 2, 8, 1 };
Array.Sort(arr);                        // {1,2,5,8}  in place
Array.Reverse(arr);                     // {8,5,2,1}
int idx = Array.IndexOf(arr, 5);        // 1
int pos = Array.BinarySearch(new[] {1,3,5}, 3);  // 1 (array must be sorted)
Array.Resize(ref arr, 6);               // creates a NEW array and copies
int[] copy = (int[])arr.Clone();        // shallow copy
Array.Fill(arr, 0);
bool any = Array.Exists(arr, x => x > 7);
```

> ⚡ **Tricky Point**
>
> Arrays are **covariant** for reference types: `object[] objs = new string[2]; objs[0] = 1;` compiles but throws `ArrayTypeMismatchException` at runtime. Generic `List<T>` is not covariant and therefore type-safe.

> ⚡ **Tricky Point**
>
> `int[] a = {1,2}; int[] b = a; b[0] = 99;` changes `a[0]` too (same object). `Clone()` makes a **shallow** copy: for an array of objects, both arrays point to the same objects.

5.6 Span<T> and Memory<T> (High-Performance Slicing)

> ✎ **Definition**
>
> Span<T> is a stack-only, type-safe view over a contiguous region of memory that lets you slice without copying. Memory<T> is its heap-storable counterpart that can be used in async code.

`Span<T>` is a **ref struct** that represents a contiguous region of memory (array, stack memory, native memory) **without copying**.

```
int[] data = { 1, 2, 3, 4, 5 };
Span<int> slice = data.AsSpan(1, 3);     // view over {2,3,4} - no allocation
slice[0] = 99;                            // data is now {1,99,3,4,5}

ReadOnlySpan<char> text = "TXN-2024-0001".AsSpan();
ReadOnlySpan<char> year = text.Slice(4, 4);      // "2024" without Substring allocation
int y = int.Parse(year);

Span<byte> buffer = stackalloc byte[256];        // stack memory, no GC
```

⚡ Tricky Point

`Span<T>` **cannot** be stored in a class field, boxed, captured in a lambda, or used across `await` (it can only live on the stack). Use `Memory<T>` when you need to store it or use it in async code.

6. Memory Management and Garbage Collection

6.1 Stack vs Heap

✎ Definition

The **stack** is a fast, per-thread LIFO memory area for method calls and local values, freed automatically when a method returns. The **heap** is a shared memory area for objects, freed by the garbage collector.

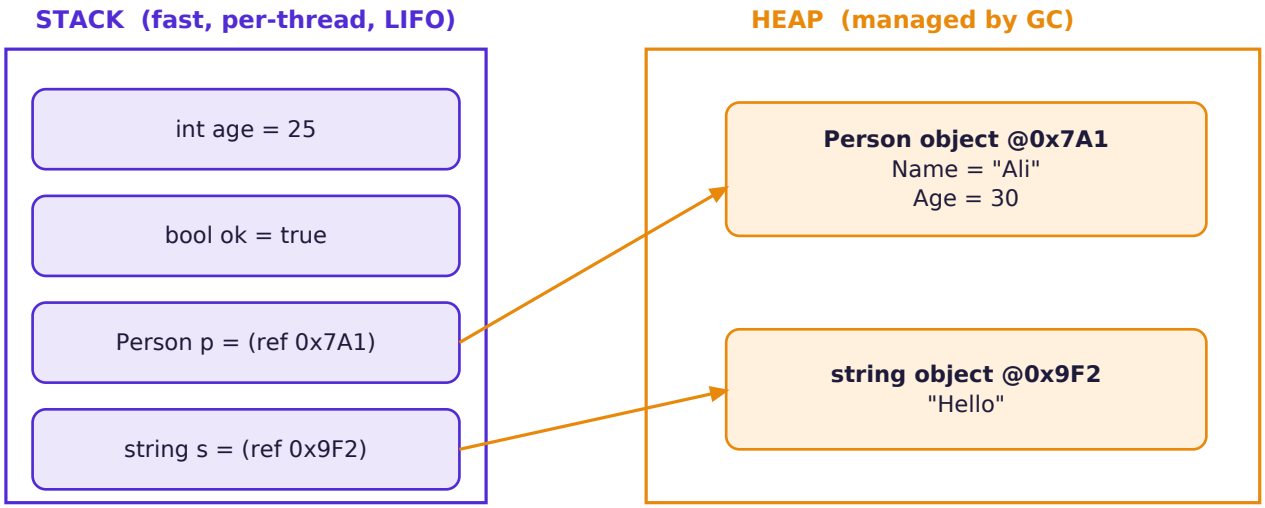


Figure 6.1 - Locals and references on the stack, objects on the heap

Stack	Heap
One per thread (default ~1 MB)	Shared by the whole process
LIFO: memory freed automatically when the method returns	Freed by the Garbage Collector
Very fast allocation (move a pointer)	Allocation is fast too, but GC has a cost
Holds local value types, references, return addresses	Holds every object created with `new` (class instances, arrays, strings, boxed values)
Overflow = StackOverflowException	Out of memory = OutOfMemoryException

☼ Simple Analogy

The stack is a pile of plates in a canteen: you put and take from the top only, very quickly. The heap is a big warehouse where boxes are placed anywhere, and a cleaner (GC) periodically removes boxes nobody has a ticket for.

6.2 How the Garbage Collector Works

> ✎ **Definition**
>
> The **Garbage Collector (GC)** automatically frees heap memory used by objects that are no longer reachable. It uses generations (0, 1, 2) and the Large Object Heap to do this efficiently.

The GC automatically frees heap objects that are **no longer reachable**. It starts from **roots** (static fields, local variables on stacks, CPU registers, GC handles) and marks every object it can reach. Everything unmarked is garbage.

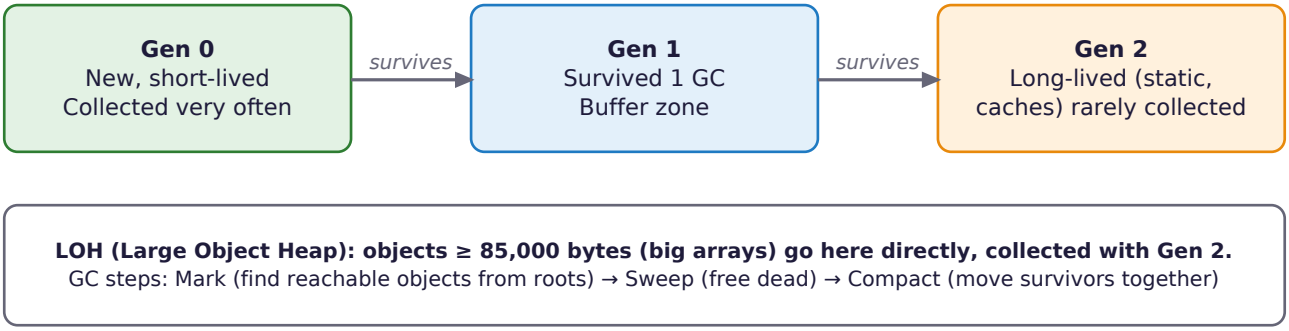


Figure 6.2 - Generational garbage collection

- **Generational hypothesis:** most objects die young. So Gen 0 is collected very often and cheaply.
- **Mark, Sweep, Compact:** live objects are marked, dead ones are freed, survivors are moved together (compaction) to avoid fragmentation.
- **LOH:** large objects (85,000+ bytes) are not compacted by default because copying them is expensive.
- **Workstation vs Server GC:** server GC uses one heap and thread per core (better throughput for ASP.NET); workstation GC is tuned for desktop responsiveness.
- **Background GC:** Gen 2 collections run concurrently with your threads to reduce pauses.

> ⚡ **Tricky Point**
>
> `GC.Collect()` forces a collection but is almost always a **bad idea** in production - it promotes live objects to older generations and disrupts the GC's self-tuning. A memory "leak" in .NET usually means something still holds a reference: static collections that only grow, event handlers that are never unsubscribed, caches without expiry, or timers.

6.3 IDisposable and the using Statement

> ✎ **Definition**
>
> `IDisposable` is the pattern for **deterministically releasing unmanaged resources** (files, connections, handles). The `using` statement calls `Dispose()` automatically, even if an exception occurs.

The GC manages **memory** only. Unmanaged resources (file handles, DB connections, sockets, OS handles) must be released explicitly - that is what `IDisposable.Dispose()` is for.

```
using (var conn = new SqlConnection(cs))      // classic using block
{
    conn.Open();
    // ...
}   // conn.Dispose() called here, even if an exception occurs

using var file = new StreamWriter("log.txt");  // using declaration (C# 8): disposed at end of scope
file.WriteLine("hello");

await using var stream = new FileStream(...);  // IAsyncDisposable
```

The compiler converts `using` to a `try/finally` that calls `Dispose()`.

Implementing the Dispose pattern

```
public class ReportWriter : IDisposable
{
    private readonly FileStream _stream;
    private bool _disposed;

    public ReportWriter(string path) => _stream = File.OpenWrite(path);

    public void Dispose()
    {
        Dispose(true);
        GC.SuppressFinalize(this);       // no need for the finalizer anymore
    }

    protected virtual void Dispose(bool disposing)
    {
        if (_disposed) return;            // safe to call Dispose many times
        if (disposing) _stream.Dispose(); // free managed resources
        // free unmanaged resources here (if any)
        _disposed = true;
    }
    ~ReportWriter() => Dispose(false);   // finalizer: only if you own UNMANAGED resources
}
```

⚡ Tricky Point

Finalizer vs Dispose: `Dispose` is deterministic (you call it). A finalizer (`~ClassName`) runs at a non-deterministic time on the finalizer thread, and objects with finalizers survive one extra GC (they are promoted). Only write a finalizer if you directly hold an unmanaged handle; prefer `SafeHandle`.

⚠ Watch Out

Do **not** wrap `HttpClient` in `using` for every request - it causes socket exhaustion. Reuse one instance (static) or use `IHttpClientFactory`.

6.4 Weak References

✎ Definition

A `WeakReference` refers to an object **without keeping it alive**, so the GC can still collect it. Useful for caches.

A `WeakReference<T>` lets you reference an object without preventing GC from collecting it - useful for caches.

```
var weak = new WeakReference<BigObject>(new BigObject());
if (weak.TryGetTarget(out var obj)) obj.Use();   // may already be collected
```

7. Object-Oriented Programming (OOP)

7.1 Classes and Objects

> ✎ **Definition**
>
> A **class** is a blueprint that defines data (fields, properties) and behaviour (methods). An **object** is an instance of a class created with new, living on the heap.

A **class** is a blueprint; an **object** is an instance created from it with `new`.

> ☼ **Simple Analogy**
>
> A class is the architect's drawing of a house; objects are the actual houses built from it. Each house has its own paint colour (state), but all share the same design (behaviour).

```
public class BankAccount
{
    // Fields - store state (keep private)
    private decimal _balance;

    // Properties - controlled access to state
    public string AccountNo { get; }
    public string Owner { get; set; }
    public decimal Balance => _balance;          // read-only computed property
    // Constructor
    public BankAccount(string accountNo, string owner)
    {
        AccountNo = accountNo;
        Owner = owner;
    }

    // Methods - behaviour
    public void Deposit(decimal amount)
    {
        if (amount <= 0) throw new ArgumentOutOfRangeException(nameof(amount));
        _balance += amount;
    }
}

var acc = new BankAccount("AC001", "Kaif");      // object creation
acc.Deposit(500);
Console.WriteLine(acc.Balance);                   // 500
```

7.2 The Four Pillars of OOP

✎ Definition

The four core principles of object-oriented design: **Encapsulation** (hide data), **Abstraction** (expose only what is essential), **Inheritance** (reuse and extend) and **Polymorphism** (one interface, many forms).

Pillar	Meaning	C# feature
Encapsulation	Hide internal state; expose safe operations	private fields, properties, access modifiers
Abstraction	Show only what is necessary, hide complexity	abstract classes, interfaces
Inheritance	Reuse and extend an existing class	`class Dog : Animal`
Polymorphism	One interface, many implementations	overloading (compile-time), virtual/override (runtime)

7.3 Access Modifiers

✎ Definition

Keywords that control **visibility** of types and members: `public`, `private`, `protected`, `internal`, `protected internal`, `private protected` and `file`.

Modifier	Same class	Derived class (same assembly)	Other class (same assembly)	Derived (other assembly)	Anywhere
`private`	Yes	No	No	No	No
`protected`	Yes	Yes	No	Yes	No
`internal`	Yes	Yes	Yes	No	No
`protected internal`	Yes	Yes	Yes	Yes	No
`private protected`	Yes	Yes	No	No	No
`public`	Yes	Yes	Yes	Yes	Yes
`file` (C# 11, types only)	Visible only in the same source file	-	-	-	-

⚡ Tricky Point

Defaults: class members are **private**; top-level classes are **internal**; interface members are **public**; enum members are public. `protected internal` = protected **OR** internal. `private protected` = protected **AND** internal.

7.4 Constructors

✎ Definition

A **constructor** is a special method with the class name and no return type. It runs when an object is created to initialise its state. Variants: default, parameterised, static, private, chained and primary constructors.

```
public class Customer
{
    public string Name { get; }
    public int Age { get; }
    public static int Count { get; private set; }

    static Customer() { Console.WriteLine("Static ctor - runs once"); }

    public Customer() : this("Unknown", 0) { }        // chaining to another ctor
    public Customer(string name) : this(name, 18) { }
    public Customer(string name, int age)
    {
        Name = name; Age = age; Count++;
    }
}
```

Constructor type	Key facts
Default (parameterless)	Compiler provides one ONLY if you write no constructor at all
Parameterised	Takes arguments to initialise state
Static	No access modifier, no parameters; runs once, automatically, before first use of the type; cannot be called directly
Private	Prevents `new` from outside: used for Singleton and static-like factories
Copy constructor	`public Customer(Customer other)` - manual copying (records generate one)

Constructor type	Key facts
Primary constructor (C# 12)	`class Service(ILogger log) { ... }` - parameters available in the whole class body

⚡ Tricky Point

Order of execution when creating a derived object: (1) derived static field initialisers, (2) derived static ctor, (3) base static field initialisers, (4) base static ctor, (5) derived instance field initialisers, (6) base instance field initialisers, (7) base constructor body, (8) derived constructor body. Note that field initialisers run *derived first*, but constructor *bodies* run *base first*.

⚠ Watch Out

Never call a **virtual** method from a base constructor: the override in the derived class runs before the derived constructor has initialised its fields.

Object initialisers

```
var c = new Customer { Nick = "Mus", City = "Pune" };    // calls ctor, then sets properties
var list = new List<int> { 1, 2, 3 };                    // collection initialiser (calls Add)
```

Primary constructors (C# 12)

```
public class OrderService(IOrderRepository repo, ILogger<OrderService> logger)
{
    public Order Get(int id)
    {
        logger.LogInformation("Fetching {Id}", id);
        return repo.Find(id);
    }
}
```

⚡ Tricky Point

Primary-constructor parameters of a class are **not** properties and **not** readonly - they are captured like mutable fields. If you need readonly, assign them to a `private readonly` field explicitly.

7.5 Properties

✎ Definition

A **property** is a member that looks like a field but uses `get`/`set` (or `init`) accessors, so access to the underlying data is controlled. Auto-properties generate the backing field for you.

```
public class Product
{
    private decimal _price;
    public decimal Price                          // full property with validation
    {
        get => _price;
        set => _price = value >= 0 ? value : throw new ArgumentException("Negative");
    }
    public string Name { get; set; } = "";        // auto-property with initialiser
    public int Id { get; init; }                  // init-only: settable only during creation (C# 9)
    public required string Sku { get; set; }      // must be set in object initialiser (C# 11)
    public string Code { get; private set; }      // public read, private write
    public decimal PriceWithTax => Price * 1.18m; // computed, read-only
    public string Description { get; set => field = value.Trim(); }  // 'field' keyword (C# 14)
}
var p = new Product { Id = 1, Sku = "X1", Name = "Pen" };
// p.Id = 2;   // error: init-only
```

★ Interview Tip

Fields vs properties: a property is a pair of methods (`get_Price`, `set_Price`) that look like a field. Properties allow validation, lazy loading, change notification, can be in interfaces and data binding works with them. Public fields break encapsulation.

7.6 static Members and Static Classes

✎ Definition

`static` members belong to the **type itself**, not to any instance, and are shared by all callers. A static class holds only static members and cannot be instantiated.

```
public static class MathHelper                // cannot be instantiated or inherited
{
    public const double Gst = 0.18;
    public static decimal AddGst(decimal amt) => amt * 1.18m;
}
public class Counter
{
    public static int Total;                   // shared by all instances
    public int Mine;                           // one per instance
    public Counter() { Total++; Mine++; }
}
new Counter(); new Counter();
Console.WriteLine(Counter.Total);   // 2
```

⚡ Tricky Point

A static method cannot access instance members (there is no `this`). Static fields are per **AppDomain/process** and per **closed generic type**: `Cache<int>.Count` and `Cache<string>.Count` are different fields.

7.7 Inheritance

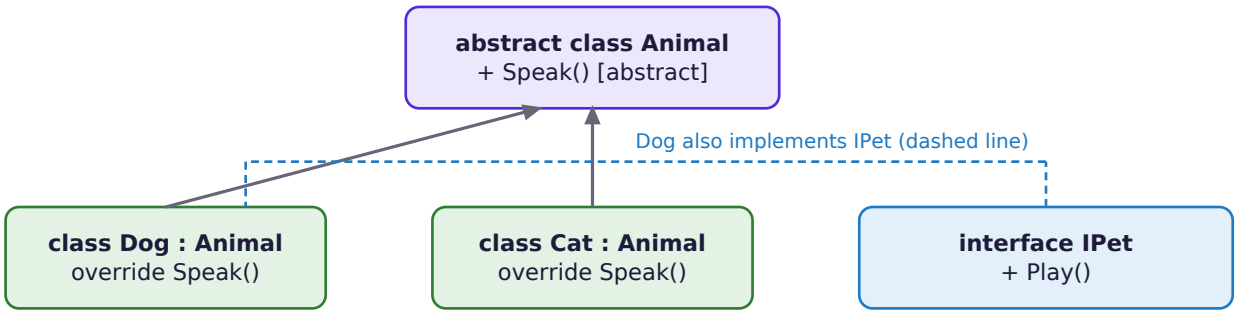
✎ Definition

Inheritance lets a class (derived) reuse and extend the members of another class (base) using `:`. C# supports single class inheritance and multiple interface implementation.

```
public class Employee
{
    public string Name { get; set; } = "";
    public virtual decimal GetSalary() => 30000;
    public Employee(string name) => Name = name;
}

public class Manager : Employee
{
    public decimal Bonus { get; set; }
    public Manager(string name, decimal bonus) : base(name)   // call base ctor
        => Bonus = bonus;
    public override decimal GetSalary() => base.GetSalary() + Bonus;  // extend base
}
```

- C# supports **single class inheritance** only (one base class), but a class can implement **many interfaces**.
- All classes ultimately derive from `System.Object`.
- `sealed class` cannot be inherited (`string` is sealed). `sealed override` stops further overriding.
- Constructors are **not** inherited.



Animal a = new Dog(); a.Speak(); → runs Dog.Speak() (runtime polymorphism)

Figure 7.1 - Inheritance, interfaces and runtime polymorphism

7.8 Polymorphism: virtual, override, new

✎ Definition

Runtime polymorphism: a `virtual` method can be `override`-n in a derived class and the call is chosen by the object's real type. new *hides* the base member instead of overriding it.

This is a favourite interview topic.

```
class Base
{
    public virtual void Show() => Console.WriteLine("Base.Show");
    public void Print() => Console.WriteLine("Base.Print");
}
class Derived : Base
{
    public override void Show() => Console.WriteLine("Derived.Show");  // overriding
    public new void Print() => Console.WriteLine("Derived.Print");     // hiding (shadowing)
}
Base b = new Derived();
b.Show();    // Derived.Show  <- decided by RUNTIME object type (virtual dispatch)
b.Print();   // Base.Print    <- decided by COMPILE-TIME variable type (no virtual)
Derived d = new Derived();
d.Print();   // Derived.Print
```

override	new (method hiding)
Requires base member to be virtual/abstract/override	Works with any base member
Replaces the method in the virtual table	Creates a separate, unrelated method
Called based on the actual object type	Called based on the reference type
True runtime polymorphism	No polymorphism

⚡ Tricky Point

If you declare a method with the same name in the derived class and use **neither** `override` nor `new`, the compiler gives warning CS0108 and treats it as `new` (hiding).

★ Interview Tip

"Can we override a static or private method?" No. Static members belong to the type (no virtual dispatch); private members are not visible to derived classes. You can only *hide* a static method with `new`.

7.9 Abstract Classes

> **✎ Definition**
>
> An `abstract` class is an **incomplete base class** that cannot be instantiated. It can contain both implemented members and abstract members that derived classes must implement.

An abstract class cannot be instantiated. It can contain both **abstract members** (no body, must be overridden) and **concrete members**.

```
public abstract class PaymentProcessor
{
    public string TxnId { get; } = Guid.NewGuid().ToString();
    protected PaymentProcessor() { }                        // constructors allowed

    public abstract bool Authorize(decimal amount);         // must override
    public virtual void Log(string m) => Console.WriteLine(m);  // may override

    public void Process(decimal amount)                     // Template Method pattern
    {
        Log($"Start {TxnId}");
        if (Authorize(amount)) Log("Approved"); else Log("Declined");
    }
}
public class CardProcessor : PaymentProcessor
{
    public override bool Authorize(decimal amount) => amount < 50000;
}
```

7.10 Interfaces

> **✎ Definition**
>
> An **interface** is a **contract**: a set of members that implementing types promise to provide. A class can implement many interfaces. Default implementations are allowed from C# 8.

An interface is a **contract**: a set of members a class promises to implement.

```
public interface IPaymentGateway
{
    string Name { get; }
    Task<bool> ChargeAsync(decimal amount);
    void Refund(string id) => Console.WriteLine("Not supported");  // default implementation (C# 8)
    static IPaymentGateway Create() => new RazorpayGateway();       // static members (C# 8/11)
}
public class RazorpayGateway : IPaymentGateway, IDisposable
{
    public string Name => "Razorpay";
    public Task<bool> ChargeAsync(decimal amount) => Task.FromResult(true);
    public void Dispose() { }
}
```

Explicit interface implementation

Used when two interfaces have the same member or you want to hide a member from the class's public surface.

```
interface IPrinter { void Print(); }
interface IScanner { void Print(); }
class Machine : IPrinter, IScanner
{
    void IPrinter.Print() => Console.WriteLine("Printing");
    void IScanner.Print() => Console.WriteLine("Scan preview");
}
var m = new Machine();
// m.Print();              // error: not accessible through class type
((IPrinter)m).Print();     // Printing
```

⚡ Tricky Point

Default interface methods are **not** visible through the class reference: `new RazorpayGateway().Refund("x")` fails to compile. You must call it through `IPaymentGateway`.

7.11 Abstract Class vs Interface

✎ Definition

Use an **abstract class** for an "is-a" relationship with shared code and state (single inheritance). Use an **interface** for a "can-do" capability that many unrelated types can implement (multiple allowed).

Abstract class	Interface
Can have fields, constructors, state	No instance fields or constructors
A class inherits only ONE	A class implements MANY
Members can be any access modifier	Members public by default (C# 8 allows others)
"is-a" relationship (Dog is an Animal)	"can-do" capability (IComparable, IDisposable)
Good for sharing common code among closely related classes	Good for contracts across unrelated classes, DI, testing/mocking
Adding a new abstract member breaks derived classes	Adding a member with a default implementation does not break implementers

★ Interview Tip

One-line answer: "Use an abstract class when related classes share **state and code**; use an interface to define a **capability/contract** that unrelated classes can implement - and because C# allows only one base class but many interfaces."

7.12 System.Object Members

✎ Definition

The members every type inherits from `System.Object`: `ToString`, `Equals`, `GetHashCode`, `GetType`, `MemberwiseClone` and `Finalize`.

Every type inherits these: `ToString()`, `Equals(object)`, `GetHashCode()`, `GetType()`, `MemberwiseClone()` (protected), `Finalize()` (protected), static `ReferenceEquals(a, b)`.

```
public class Money
{
    public decimal Amount { get; }
    public string Currency { get; }
    public Money(decimal a, string c) { Amount = a; Currency = c; }

    public override string ToString() => $"{Amount:N2} {Currency}";
    public override bool Equals(object? obj) =>
        obj is Money m && m.Amount == Amount && m.Currency == Currency;
    public override int GetHashCode() => HashCode.Combine(Amount, Currency);
}
```

⚡ Tricky Point

Rule: if you override `Equals`, you must override `GetHashCode` consistently - equal objects must return equal hash codes. Otherwise `Dictionary` and `HashSet` will "lose" your objects. Never use a mutable property in `GetHashCode` for objects stored as dictionary keys.

== vs Equals vs ReferenceEquals

Expression	Classes (default)	Strings	Value types / records
`a == b`	Reference equality (unless overloaded)	Content (overloaded)	Structs: not defined unless overloaded; records: value
`a.Equals(b)`	Reference equality (unless overridden)	Content	Field-by-field value equality
`ReferenceEquals(a,b)`	Same object?	Same object?	Always false for value types (boxing)

7.13 Shallow Copy vs Deep Copy

✎ Definition

A **shallow copy** copies an object's fields, so nested references are shared with the original. A **deep copy** also clones every referenced object, giving a fully independent copy.

```
public class Order
{
    public int Id;
    public List<string> Items = new();
    public Order ShallowCopy() => (Order)MemberwiseClone();
    public Order DeepCopy()
    {
        var copy = (Order)MemberwiseClone();
        copy.Items = new List<string>(Items);      // copy referenced objects too
        return copy;
    }
}
var o1 = new Order { Id = 1 }; o1.Items.Add("Pen");
var o2 = o1.ShallowCopy();
o2.Items.Add("Book");     // o1.Items ALSO has "Book" (shared list)
```

7.14 Partial Classes and Partial Methods

> **✎ Definition**
>
> `partial` lets one class, struct or interface be **split across multiple files** that the compiler merges. A partial method is declared in one part and implemented in another (common in generated code).

Splits one class across multiple files - used by designers and source generators.

```
// File1.cs
public partial class Invoice { public int Id { get; set; } partial void OnCreated(); }
// File2.cs
public partial class Invoice { partial void OnCreated() => Console.WriteLine("Created"); }
```

7.15 Nested Classes and the this Keyword

> **✎ Definition**
>
> A **nested class** is a type declared inside another type, with access to its private members. `this` refers to the current instance (also used for constructor chaining, indexers and extension methods).

```
public class Outer
{
    private int _secret = 42;
    public class Inner { public int Read(Outer o) => o._secret; }   // can access private members
}
public class Point
{
    private int x;
    public Point(int x) { this.x = x; }        // 'this' disambiguates field vs parameter
    public Point Move(int dx) { x += dx; return this; }   // fluent API
}
```

7.16 Operator Overloading and Conversion Operators

> **✎ Definition**
>
> **Operator overloading** defines what operators such as + or == do for your own types. **Conversion operators** (`implicit`/`explicit`) define casts to and from them.

```
public readonly struct Money
{
    public decimal Value { get; }
    public Money(decimal v) => Value = v;
    public static Money operator +(Money a, Money b) => new(a.Value + b.Value);
    public static bool operator ==(Money a, Money b) => a.Value == b.Value;
    public static bool operator !=(Money a, Money b) => !(a == b);   // must overload in pairs
    public static implicit operator Money(decimal d) => new(d);      // decimal -> Money automatically
    public static explicit operator decimal(Money m) => m.Value;     // requires a cast
    public override bool Equals(object? o) => o is Money m && m == this;
    public override int GetHashCode() => Value.GetHashCode();
}
Money total = 100m;                 // implicit
total = total + 50m;
decimal raw = (decimal)total;       // explicit
```

7.17 Indexers

✎ Definition

An **indexer** lets objects of a class be accessed with array syntax (`obj[i]`). It is declared with `this[...]` and get/set accessors.

```
public class Matrix
{
    private readonly double[,] _d = new double[3, 3];
    public double this[int r, int c] { get => _d[r, c]; set => _d[r, c] = value; }
}
var m = new Matrix(); m[0, 1] = 5;
```

7.18 SOLID Principles (Quick Guide)

✎ Definition

SOLID is five design principles for maintainable OO code: Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation and Dependency Inversion.

Principle	Meaning	Example
S - Single Responsibility	A class should have one reason to change	Split `InvoiceService` and `InvoicePdfExporter`
O - Open/Closed	Open for extension, closed for modification	Add new `IFeeRule` classes instead of editing a giant switch
L - Liskov Substitution	Derived types must be usable wherever the base is expected	A `ReadOnlyFile` that throws on `Write()` violates LSP
I - Interface Segregation	Many small interfaces beat one fat interface	`IReader` + `IWriter` instead of `IFileManager` with 20 methods
D - Dependency Inversion	Depend on abstractions, not concretions	Constructor takes `IRepository`, not `SqlRepository`

8. Structs, Records, Enums, Tuples and Anonymous Types

8.1 Structs

> ✎ **Definition**
>
> A `struct` is a lightweight **value type** for small data. It is copied on assignment, cannot inherit from another struct or class, and can implement interfaces.

A struct is a **value type**: lightweight, copied on assignment, no inheritance (but can implement interfaces).

```
public struct Point
{
    public int X { get; set; }
    public int Y { get; set; }
    public Point(int x, int y) { X = x; Y = y; }
    public double Distance() => Math.Sqrt(X * X + Y * Y);
}
Point p1 = new Point(3, 4);
Point p2 = p1;     // full copy
p2.X = 100;        // p1.X is still 3
public readonly struct Coordinate(double Lat, double Lng);   // immutable struct (primary ctor C# 12)
```

class	struct
Reference type (heap)	Value type (inline / stack)
Supports inheritance	No inheritance (implicitly sealed, derives from ValueType)
Can be null	Cannot be null (unless `Nullable<T>`)
Copy = copy reference	Copy = copy all data
Default equality: reference	Default `Equals`: field-by-field (via reflection - slow; override it)
Use for entities, services, larger objects	Use for small (< 16 bytes guideline), immutable, short-lived values: Point, Money, DateTime

> ⚡ **Tricky Point**
>
> **Mutable struct trap**: `list[0].X = 5;` on a `List<Point>` gives a compile error because the indexer returns a **copy**. With a `readonly` field holding a mutable struct, calling a mutating method silently mutates a defensive copy. Make structs `readonly` whenever possible.

> ⚡ **Tricky Point**
>
> `default(Point)` and `new Point()` always exist (all fields zero), even if you declare other constructors. Since C# 10 you can write your own parameterless struct constructor, but `default(Point)` still ignores it.

ref struct

`ref struct` types (like `Span<T>`) can **only** live on the stack: they can't be boxed, be fields of classes, be captured by lambdas or (until C# 13) implement interfaces.

8.2 Records (C# 9+)

✎ Definition

A `record` is a type built for **immutable data**, with compiler-generated **value-based equality**, `ToString`, deconstruction and non-destructive mutation using `with`.

Records are types designed for **immutable data with value-based equality**.

```
public record Person(string Name, int Age);            // positional record (class)

var p1 = new Person("Ali", 30);
var p2 = new Person("Ali", 30);
Console.WriteLine(p1 == p2);          // True  - value equality!
Console.WriteLine(p1);                // Person { Name = Ali, Age = 30 }  - built-in ToString
var p3 = p1 with { Age = 31 };        // non-destructive mutation (copy + change)
var (name, age) = p1;                 // built-in Deconstruct

public record struct Point(int X, int Y);      // value-type record (mutable by default)
public readonly record struct Rgb(byte R, byte G, byte B);

public record Employee(string Name, decimal Salary) : Person(Name, 0)   // records support inheritance
{
    public string Dept { get; init; } = "IT";       // extra members
}
```

The compiler generates for a record: init-only properties, constructor, `Equals`, `GetHashCode`, `==`/`!=`, `ToString`, `Deconstruct`, a copy constructor and `<Clone>$` (used by `with`).

class	record (class)	struct	record struct
Reference equality	Value equality	Value equality (slow default)	Value equality (fast, generated)
Mutable by default	Immutable by default (init)	Mutable	Mutable unless `readonly`
Heap	Heap	Stack/inline	Stack/inline

⚡ Tricky Point

`with` creates a **shallow** copy. If the record holds a `List<T>`, both the original and the copy share the same list. Also, value equality for a record containing a `List<T>` compares the list *references*, not contents.

★ Interview Tip

Use records for DTOs, API request/response models, events, value objects and configuration snapshots. Use classes for entities with identity and behaviour (EF Core entities are usually classes).

8.3 Enums

✎ Definition

An enum is a value type that defines a set of **named integral constants**, making code readable. [Flags] enums combine values with bitwise operators.

```
public enum TxnStatus { Pending, Success, Failed = 10, Reversed }   // 0, 1, 10, 11
public enum Channel : byte { Atm = 1, Pos = 2, Ecom = 3 }            // underlying type

TxnStatus s = TxnStatus.Success;
int code = (int)s;                                  // 1
TxnStatus parsed = Enum.Parse<TxnStatus>("Failed");
bool ok = Enum.TryParse("success", ignoreCase: true, out TxnStatus st);
string[] names = Enum.GetNames<TxnStatus>();
bool defined = Enum.IsDefined(typeof(TxnStatus), 5);  // false
TxnStatus weird = (TxnStatus)999;                     // allowed! no exception
```

Flags enums (bit fields)

```
[Flags]
public enum Permission { None = 0, Read = 1, Write = 2, Delete = 4, Admin = Read | Write | Delete }

var p = Permission.Read | Permission.Write;
Console.WriteLine(p);                           // "Read, Write"
bool canWrite = p.HasFlag(Permission.Write);    // true
bool canDel = (p & Permission.Delete) != 0;     // false
p &= ~Permission.Write;                         // remove a flag
```

⚡ Tricky Point

Any integer can be cast to an enum even if not defined - always validate input with `Enum.IsDefined` or a switch with a default. Flags values must be powers of two (1, 2, 4, 8...).

8.4 Tuples

✎ Definition

A **tuple** (ValueTuple) is a lightweight value type that groups several values, optionally named, without declaring a class: `(int Id, string Name)`.

```
var t1 = Tuple.Create(1, "a");          // System.Tuple: class, immutable, Item1/Item2 - old
(int Id, string Name) t2 = (1, "Ali");  // ValueTuple: struct, mutable, named elements - modern
var t3 = (Id: 2, Name: "Sam");
Console.WriteLine(t2.Name);
var (id, nm) = t3;                       // deconstruct
(a, b) = (b, a);                         // swap without temp variable
Console.WriteLine((1, 2) == (1, 2));     // True (C# 7.3 tuple equality)
```

⚡ Tricky Point

Tuple element names are compile-time only. At runtime they're just `Item1`, `Item2`, so names don't survive reflection or JSON serialization. For public APIs prefer a record.

8.5 Anonymous Types

> ✎ **Definition**
>
> An **anonymous type** is a read-only, compiler-generated class created inline with `new { Name = "A", Age = 3 }`. Mostly used for LINQ projections within one method.

```
var anon = new { Id = 1, Name = "Pen", Price = 10m };
Console.WriteLine(anon.Name);
var proj = products.Select(p => new { p.Name, Total = p.Price * p.Qty });  // common in LINQ
```

Anonymous types are immutable reference types with compiler-generated value equality. They cannot be returned from a method with a meaningful type (only as `object`/`dynamic`) - use a record or tuple instead.

8.6 Deconstruction for Your Own Types

> ✎ **Definition**
>
> **Deconstruction** unpacks an object into separate variables: `var (x, y) = point;`. Custom types support it by defining a `Deconstruct` method with `out` parameters.

```
public class Txn
{
    public string Id { get; set; } = "";
    public decimal Amount { get; set; }
    public void Deconstruct(out string id, out decimal amount) => (id, amount) = (Id, Amount);
}
var (txnId, amt) = new Txn { Id = "T1", Amount = 50 };
```

9. Exception Handling

9.1 What is an Exception?

> ✎ **Definition**
>
> An **exception** is an object that represents an error raised at run time. It interrupts normal flow and travels up the call stack until a matching handler catches it.

An exception is an object representing an error that happens at runtime. When thrown, the CLR **unwinds the call stack** looking for a matching `catch` block. If none is found, the application terminates.

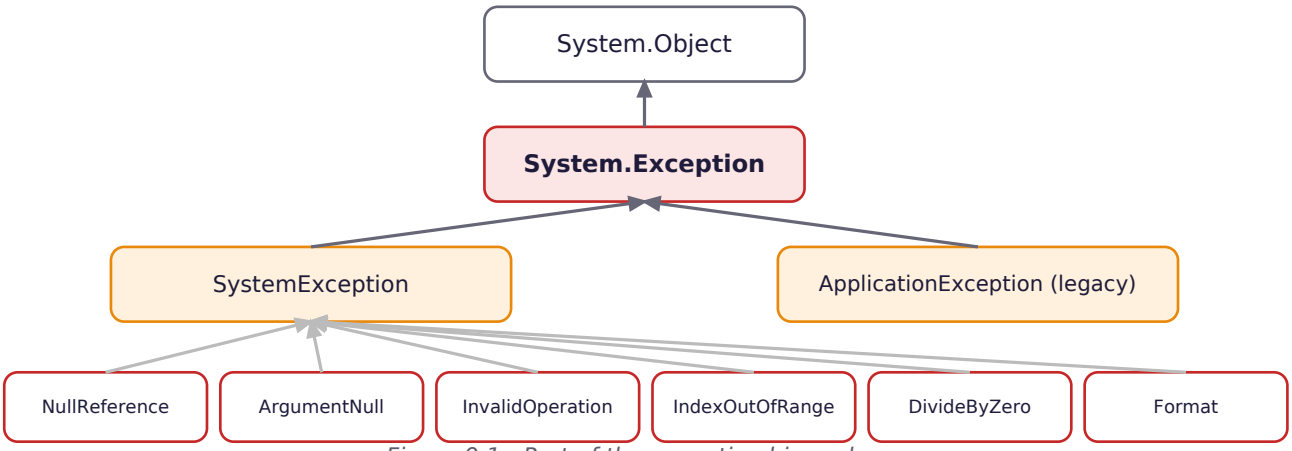


Figure 9.1 - Part of the exception hierarchy

9.2 try, catch, finally

✎ Definition

`try` wraps code that may throw, `catch` handles specific exception types, and `finally` always runs for cleanup, whether or not an exception occurred.

```
try
{
    var amount = decimal.Parse(input);
    var result = 100 / divisor;
}
catch (FormatException ex)                  // most specific first
{
    Console.WriteLine($"Bad number: {ex.Message}");
}
catch (DivideByZeroException)
{
    Console.WriteLine("Divisor was zero");
}
catch (Exception ex) when (ex is IOException || ex is TimeoutException)  // exception filter
{
    Log(ex);
}
catch (Exception ex)                        // most general last
{
    Log(ex);
    throw;                                  // rethrow, keeping the stack trace
}
finally
{
    Console.WriteLine("Always runs - cleanup here");
}
```

⚡ Tricky Point

Catch blocks are checked **top to bottom**. Putting `catch (Exception)` before `catch (FormatException)` is a compile error (CS0160) because the second can never be reached.

9.3 throw vs throw ex

✎ Definition

`throw;` re-throws the current exception and **keeps the original stack trace**. `throw ex;` resets the stack trace to the current line, which loses debugging information.

```
catch (Exception ex)
{
    throw;           // GOOD: preserves original stack trace
    throw ex;        // BAD: resets stack trace to this line - you lose where it really failed
    throw new BusinessException("Settlement failed", ex);   // GOOD: wrap with InnerException
}
```

★ Interview Tip

"Difference between throw and throw ex?" - `throw;` rethrows the same exception object and keeps the full original stack trace. `throw ex;` resets the stack trace to the current method, hiding the original source of the error.

9.4 Tricky Cases with finally

> **✎ Definition**
>
> Edge cases of `finally`: it runs even after `return`, it cannot change an already-evaluated return value, and it is skipped only in extreme cases such as process termination.

```
int Test()
{
    int x = 1;
    try { return x; }          // return value (1) is captured here
    finally { x = 99; }        // runs, but does not change the captured value
}
Console.WriteLine(Test());    // 1

string Test2()
{
    try { throw new Exception("A"); }
    finally { Console.WriteLine("finally runs before caller's catch"); }
}
```

- You **cannot** `return` from a `finally` block (compile error).
- `finally` does **not** run if the process is killed, on `Environment.FailFast`, or on `StackOverflowException`.
- An exception thrown inside `finally` replaces (hides) the original exception.

9.5 Exception Filters (when)

> **✎ Definition**
>
> A **filter** (`catch (Ex e) when (condition)`) catches an exception only if the condition is true. Otherwise the stack is not unwound and the search for a handler continues.

Filters are evaluated **before** the stack unwinds, so the original context is preserved for debugging, and a false filter doesn't catch the exception at all.

```
catch (HttpRequestException ex) when (ex.StatusCode == HttpStatusCode.NotFound) { return null; }
catch (SqlException ex) when (ex.Number == 1205) { /* deadlock - retry */ }
catch (Exception ex) when (LogAndReturnFalse(ex)) { }   // log without catching
```

9.6 Custom Exceptions

> **✎ Definition**
>
> A **custom exception** is your own class derived from `Exception`. It represents a domain-specific error and carries extra details.

```
public class InsufficientFundsException : Exception
{
    public decimal Shortfall { get; }
    public InsufficientFundsException() { }
    public InsufficientFundsException(string message) : base(message) { }
    public InsufficientFundsException(string message, Exception inner) : base(message, inner) { }
    public InsufficientFundsException(decimal shortfall)
        : base($"Insufficient funds. Short by {shortfall}") => Shortfall = shortfall;
}
throw new InsufficientFundsException(250m);
```

Naming convention: end with `Exception`; derive from `Exception` (not `ApplicationException`); provide the three standard constructors.

9.7 Common Built-in Exceptions

> **✎ Definition**
>
> Standard exception types in .NET, such as `ArgumentNullException`, `InvalidOperationException`, `NullReferenceException`, `IndexOutOfRangeException` and `KeyNotFoundException`.

Exception	When it happens
`NullReferenceException`	Accessing a member on a null reference
`ArgumentNullException`	A method received null for a required argument
`ArgumentOutOfRangeException`	Argument outside allowed range (e.g. list index)
`ArgumentException`	Generic invalid argument
`InvalidOperationException`	Object state invalid for the call (e.g. `First()` on empty, modifying collection in foreach)
`IndexOutOfRangeException`	Bad array index
`KeyNotFoundException`	Dictionary indexer with a missing key
`InvalidCastException`	Invalid explicit cast / unboxing
`FormatException`	Parse of an invalid string
`DivideByZeroException`	Integer/decimal division by zero
`OverflowException`	Arithmetic overflow in a `checked` context
`NotImplementedException` / `NotSupportedException`	Feature missing / not allowed
`OperationCanceledException` / `TaskCanceledException`	Cancellation token triggered
`TimeoutException`, `IOException`, `UnauthorizedAccessException`	I/O and system problems
`AggregateException`	Wraps multiple exceptions (Task.Wait, Parallel)

9.8 Guard Clauses (modern helpers)

> **✎ Definition**
>
> **Guard clauses** validate inputs at the start of a method and fail fast. Modern helpers such as `ArgumentNullException.ThrowIfNull` and `ArgumentException.ThrowIfNullOrEmpty` do this in one line.

```
public void Transfer(Account from, decimal amount)
{
    ArgumentNullException.ThrowIfNull(from);                // .NET 6
    ArgumentOutOfRangeException.ThrowIfNegativeOrZero(amount); // .NET 8
    ArgumentException.ThrowIfNullOrWhiteSpace(from.Id);       // .NET 8
}
```

9.9 Best Practices

✎ Definition

Proven rules for exception handling: catch specific exceptions, don't use exceptions for control flow, never swallow errors silently, preserve stack traces, and clean up with `using`/`finally`.

- Catch only exceptions you can **handle meaningfully**; let others bubble up to a global handler (middleware in ASP.NET Core).
- Never use exceptions for normal control flow - they are expensive (stack trace capture). Use `TryParse`, `TryGetValue`, etc.
- Never swallow exceptions silently: `catch { }` hides bugs.
- Always log the **full** exception (`ex.ToString()`), not only `ex.Message` - it includes inner exceptions and stack trace.
- Use `using`/`finally` for cleanup; don't rely on catch blocks.

⚡ Tricky Point

In `async` code, exceptions are stored in the returned `Task` and rethrown at `await`. `await` throws the **first** inner exception, while `.Wait()` / `.Result` throws an `AggregateException` wrapping them.

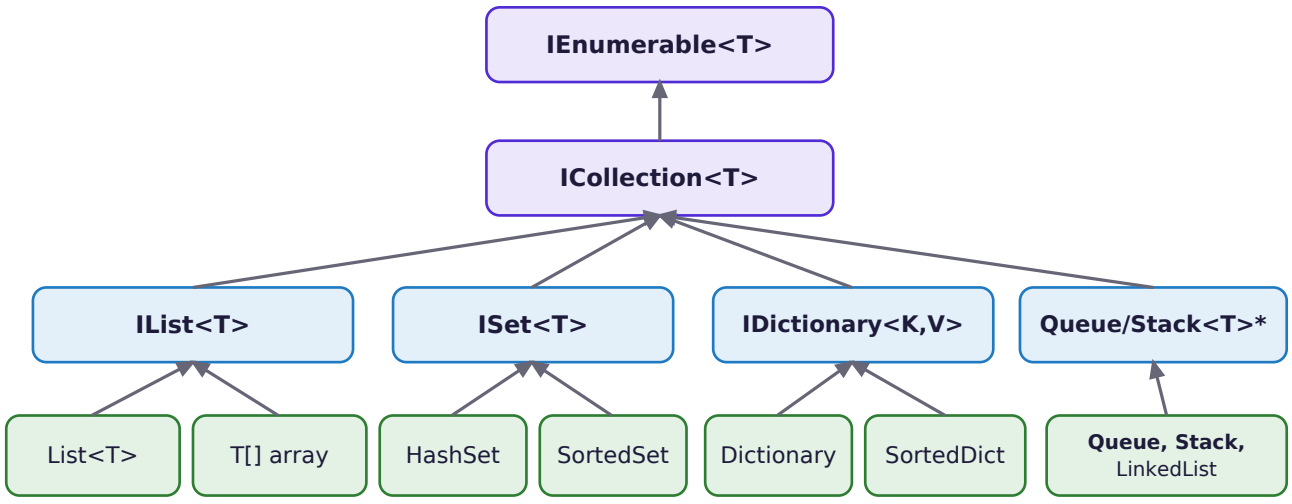
10. Collections (Complete Guide)

10.1 Overview and Hierarchy

> ✎ **Definition**
>
> **Collections** are classes that store and manage groups of objects. They are built on interfaces such as `IEnumerable<T>`, `ICollection<T>`, `IList<T>` and `IDictionary<K,V>` in `System.Collections.Generic`.

Collections store groups of objects. Unlike arrays, most collections **grow automatically**. They live in three main namespaces:

- `System.Collections.Generic` - type-safe generic collections (**use these**): `List<T>`, `Dictionary<TKey,TValue>`, `HashSet<T>`, `Queue<T>`, `Stack<T>`, `LinkedList<T>`, `SortedList`, `SortedDictionary`, `SortedSet`, `PriorityQueue`.
- `System.Collections` - old non-generic collections storing `object`: `ArrayList`, `Hashtable`, `Queue`, `Stack` (avoid in new code).
- `System.Collections.Concurrent` - thread-safe: `ConcurrentDictionary`, `ConcurrentQueue`, `ConcurrentStack`, `ConcurrentBag`, `BlockingCollection`.
- `System.Collections.Immutable` - `ImmutableList`, `ImmutableDictionary`, `ImmutableArray`, ...
- `System.Collections.Frozen` (.NET 8) - `FrozenDictionary`, `FrozenSet`: read-only, ultra-fast lookups.



* Queue<T>/Stack<T> implement IEnumerable<T> + IReadOnlyCollection<T> (not ICollection<T>).

Figure 10.1 - Key collection interfaces and implementations

The core interfaces

Interface	What it adds	Key members
`IEnumerable<T>`	Can be iterated with foreach (forward only)	`GetEnumerator()`
`ICollection<T>`	Count + modify	`Count, Add, Remove, Contains, Clear, CopyTo`
`IList<T>`	Index-based access	`this[int], IndexOf, Insert, RemoveAt`
`IDictionary<K,V>`	Key-value pairs	`this[key], Keys, Values, ContainsKey, TryGetValue`
`ISet<T>`	Unique items + set operations	`UnionWith, IntersectWith, ExceptWith, IsSubsetOf`

Interface	What it adds	Key members
`IReadOnlyCollection<T>` / `IReadOnlyList<T>` / `IReadOnlyDictionary<K,V>`	Read-only views	`Count`, indexer

> **★ Interview Tip**
>
> Return the **least capable interface** that satisfies the caller: return `IEnumerable<T>` or `IReadOnlyList<T>` from methods instead of `List<T>`, so callers can't accidentally modify your internal list. Accept parameters as `IEnumerable<T>` when you only need to loop.

10.2 List<T>

> **✎ Definition**
>
> `List<T>` is a **dynamic array**: an ordered, index-based collection that grows automatically. Index access and adding at the end are fast (O(1)).

A dynamic array: index-based, ordered, allows duplicates and nulls. **The most used collection.**

```
var txns = new List<string>();                    // empty
var nums = new List<int> { 5, 3, 8, 1 };          // collection initialiser
var big = new List<int>(capacity: 10_000);        // pre-size when you know the count
List<int> modern = [1, 2, 3];                     // C# 12 collection expression

// ADD
nums.Add(10);                          // {5,3,8,1,10}
nums.AddRange(new[] { 20, 30 });
nums.Insert(0, 99);                    // insert at index 0 - shifts everything: O(n)
nums.InsertRange(1, new[] { 7, 7 });
// READ
int first = nums[0];                   // 99
int count = nums.Count;                // NOT .Length
bool has = nums.Contains(8);           // O(n) linear search
int idx = nums.IndexOf(8);             // -1 if not found
int found = nums.Find(x => x > 6);     // first match or default(T) (0 for int!)
List<int> all = nums.FindAll(x => x > 6);
int fi = nums.FindIndex(x => x == 3);
bool any = nums.Exists(x => x < 0);
bool every = nums.TrueForAll(x => x > 0);
List<int> part = nums.GetRange(1, 3); // copy of 3 items from index 1

// REMOVE
nums.Remove(7);                        // removes FIRST occurrence; returns bool
nums.RemoveAt(0);                      // by index
nums.RemoveAll(x => x > 25);           // returns number removed
nums.RemoveRange(0, 2);
nums.Clear();                          // Count = 0 (Capacity stays)
// SORT / ORDER
var list = new List<int> { 4, 1, 3 };
list.Sort();                           // in place ascending: {1,3,4}
list.Sort((a, b) => b.CompareTo(a));   // descending with Comparison delegate
list.Reverse();
int pos = list.BinarySearch(3);        // only on sorted list: O(log n)

// CONVERT
int[] arr = list.ToArray();
var ro = list.AsReadOnly();            // ReadOnlyCollection<int> wrapper
list.ForEach(x => Console.WriteLine(x));
List<string> strs = list.ConvertAll(x => x.ToString());
```

Sorting a list of objects

```
var people = new List<Person> { new("Zara", 25), new("Ali", 30), new("Sam", 25) };
people.Sort((a, b) => a.Age.CompareTo(b.Age));                    // in place, NOT stable
var sorted = people.OrderBy(p => p.Age).ThenBy(p => p.Name).ToList(); // LINQ: new list, stable
```

How List<T> grows internally

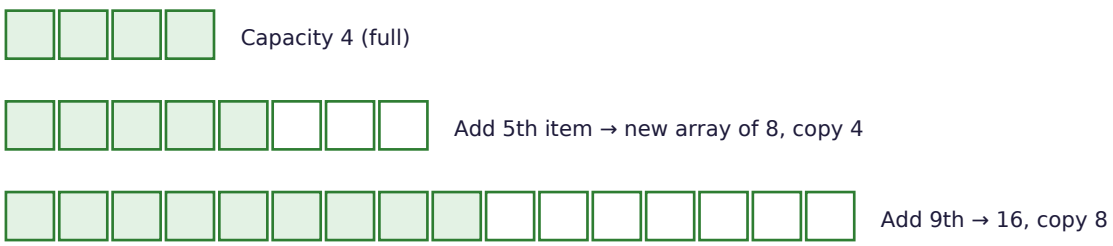


Figure 10.2 - List<T> doubles its internal array when full

`List<T>` wraps a private array. **Count** = items used; **Capacity** = array size. When Count reaches Capacity, it allocates a **new array of double size** and copies everything. That's why `Add` is *amortised* O(1) but an individual Add can be O(n).

```
var l = new List<int>();
Console.WriteLine(l.Capacity);   // 0
l.Add(1); Console.WriteLine(l.Capacity);   // 4
for (int i = 0; i < 4; i++) l.Add(i);
Console.WriteLine(l.Capacity);   // 8
l.TrimExcess();                  // shrink capacity to Count
```

⚡ Tricky Point

`List<T>.Find` returns `default(T)` when nothing matches - for `List<int>` that's **0**, indistinguishable from a real 0. Use `FindIndex` or `Exists`. Also, `Remove` in a loop and `Insert(0, x)` are O(n) each - for frequent front inserts use `LinkedList<T>` or `Queue<T>`.

⚡ Tricky Point

`List.Sort()` is **unstable** (QuickSort/IntroSort): equal elements may change order. LINQ `OrderBy` is **stable**.

10.3 Dictionary<TKey, TValue>

✎ Definition

Dictionary<TKey,TValue> is a **hash table** of unique keys mapped to values, with near O(1) lookup, insert and delete by key.

A hash table of unique keys mapped to values. **O(1) average** lookup, insert and delete. Order of enumeration is **not guaranteed**.

```
var stock = new Dictionary<string, int>
{
    ["Pen"] = 10,                    // index initialiser
    ["Book"] = 5
};
var codes = new Dictionary<string, string>(StringComparer.OrdinalIgnoreCase); // case-insensitive keys

// ADD / UPDATE
stock.Add("Bag", 3);                 // throws ArgumentException if key exists!
stock["Bag"] = 4;                    // add OR overwrite - never throws
```

```
bool added = stock.TryAdd("Pen", 99); // false, existing value kept

// READ
int pens = stock["Pen"];             // throws KeyNotFoundException if missing!
if (stock.TryGetValue("Ink", out int ink)) Console.WriteLine(ink);   // BEST way
int qty = stock.GetValueOrDefault("Ink", 0);
bool hasKey = stock.ContainsKey("Pen");     // O(1)
bool hasVal = stock.ContainsValue(5);       // O(n) - scans all values

// REMOVE
stock.Remove("Bag");
stock.Remove("Book", out int removedQty);

// ITERATE
foreach (KeyValuePair<string, int> kv in stock) Console.WriteLine($"{kv.Key}: {kv.Value}");
foreach (var (item, q) in stock) Console.WriteLine($"{item}: {q}");   // deconstruction
foreach (var key in stock.Keys) { }
foreach (var val in stock.Values) { }
```

Common real-world patterns

```
// 1) Counting / frequency map
var freq = new Dictionary<char, int>();
foreach (char c in "banana")
    freq[c] = freq.TryGetValue(c, out var n) ? n + 1 : 1;     // b:1 a:3 n:2

// 2) Grouping into lists
var byStatus = new Dictionary<string, List<Txn>>();
foreach (var t in txns)
{
    if (!byStatus.TryGetValue(t.Status, out var bucket))
        byStatus[t.Status] = bucket = new List<Txn>();
    bucket.Add(t);
}

// 3) Fast lookup instead of nested loops: O(n*m) -> O(n+m)
var bankById = bankRecords.ToDictionary(b => b.Rrn);
foreach (var sw in switchRecords)
    if (bankById.TryGetValue(sw.Rrn, out var match)) Reconcile(sw, match);

// 4) Update in place without double lookup (.NET 6+)
ref int slot = ref CollectionsMarshal.GetValueRefOrAddDefault(freq, 'z', out bool exists);
slot++;
```

How Dictionary works internally

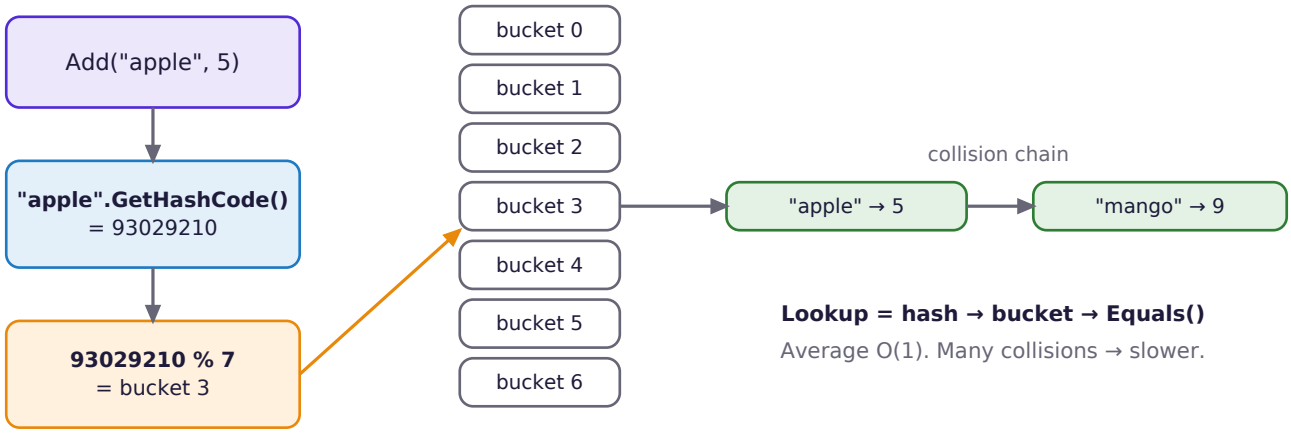


Figure 10.3 - Hashing a key to a bucket

1. `key.GetHashCode()` produces an int. 2. The hash is mapped to a **bucket** index (hash modulo bucket count, a prime number). 3. Entries in the same bucket form a chain; the key is confirmed with `Equals()`. 4. When it fills up, it **resizes** (roughly doubles to the next prime) and rehashes all entries.

> ⚡ **Tricky Point**
>
> Requirements for a good key: `GetHashCode` and `Equals` must be **consistent** and the key must be **immutable** while in the dictionary. If you mutate a key object after inserting it, its hash changes and `ContainsKey` returns false even though it's "inside". A null key is never allowed (`ArgumentNullException`).

> **⚡ Tricky Point**
>
> Dictionary enumeration order *looks* like insertion order if you never remove, but after removals new items fill freed slots. **Never rely on order** - use `SortedDictionary` or an `OrderedDictionary<TKey,TValue>` (.NET 9) if order matters.

10.4 HashSet<T>

> **✎ Definition**
>
> HashSet<T> is an unordered collection of **unique** elements, with O(1) membership checks and set operations (union, intersect, except).

A collection of **unique** values with O(1) `Add`, `Remove`, `Contains`. No index, no order. Perfect for "have I seen this?" checks and set mathematics.

```
var seen = new HashSet<string>();
bool isNew = seen.Add("TXN1");    // true
isNew = seen.Add("TXN1");         // false - duplicate silently ignored (no exception)

var a = new HashSet<int> { 1, 2, 3, 4 };
var b = new HashSet<int> { 3, 4, 5 };
a.UnionWith(b);           // a = {1,2,3,4,5}   (modifies a!)
a.IntersectWith(b);       // a = {3,4,5}
a.ExceptWith(b);          // remove items of b
a.SymmetricExceptWith(b); // items in either but not both
bool sub = a.IsSubsetOf(b);
bool overlap = a.Overlaps(b);

// Remove duplicates from a list
var unique = new HashSet<int>(new[] { 1, 1, 2, 3, 3 }).ToList();   // {1,2,3}
// Find duplicates
var dupes = list.Where(x => !seen.Add(x)).ToList();
```

> **⚡ Tricky Point**
>
> `List.Contains` is O(n) - checking 100,000 IDs against a 100,000-item List = 10 billion comparisons. Converting the list to a `HashSet` first makes it 100,000 O(1) lookups. This is a very common performance fix.

10.5 SortedSet<T>, SortedDictionary<K,V>, SortedList<K,V>

> **✎ Definition**
>
> **Sorted collections** keep elements ordered by key or value automatically. `SortedSet` and `SortedDictionary` use red-black trees; `SortedList` uses sorted arrays.

```
var ss = new SortedSet<int> { 5, 1, 9, 3 };      // iterates 1,3,5,9 - unique + sorted
Console.WriteLine(ss.Min + " " + ss.Max);        // 1 9
var view = ss.GetViewBetween(2, 6);              // {3,5}

var sd = new SortedDictionary<string, decimal> { ["Zeta"] = 1, ["Alpha"] = 2 };
foreach (var kv in sd) Console.WriteLine(kv.Key);   // Alpha, Zeta (sorted by key)

var sl = new SortedList<int, string> { [3] = "c", [1] = "a" };
Console.WriteLine(sl.Keys[0]);                   // 1 - index access available
```

SortedDictionary	SortedList
Red-black tree	Two sorted arrays (keys, values)
Insert/Remove O(log n)	Insert/Remove O(n) (shifting)
More memory per item	Less memory
No index access	Index access via `Keys[i]`, `Values[i]`
Better when data arrives unsorted and changes often	Better when data is loaded once / already sorted

10.6 Queue<T> - FIFO (First In, First Out)

✎ Definition

Queue<T> is a **First-In-First-Out** collection: Enqueue adds at the back, and Dequeue removes from the front.

```
var jobs = new Queue<string>();
jobs.Enqueue("File1");          // add to back
jobs.Enqueue("File2");
string next = jobs.Peek();      // "File1" - look without removing
string done = jobs.Dequeue();   // "File1" - remove from front
if (jobs.TryDequeue(out var j)) Console.WriteLine(j);   // safe, no exception when empty
Console.WriteLine(jobs.Count);
```

☼ Simple Analogy

A queue is a ticket counter line: first person in line is served first. Use it for job processing, BFS traversal, buffering messages.

10.7 Stack<T> - LIFO (Last In, First Out)

✎ Definition

Stack<T> is a **Last-In-First-Out** collection: Push adds to the top, and Pop removes from the top.

```
var history = new Stack<string>();
history.Push("Page1");
history.Push("Page2");
Console.WriteLine(history.Peek());   // Page2
history.Pop();                        // removes Page2
history.TryPop(out var page);         // Page1

// Classic use: balanced brackets
bool IsBalanced(string s)
{
```

```
    var st = new Stack<char>();
    foreach (var c in s)
    {
        if (c is '(' or '[' or '{') st.Push(c);
        else if (c is ')' or ']' or '}')
        {
            if (st.Count == 0) return false;
            var open = st.Pop();
            if ((open, c) is not (('(', ')') or ('[', ']') or ('{', '}'))) return false;
        }
    }
    return st.Count == 0;
}
```

⚡ Tricky Point

`Dequeue()`/`Pop()`/`Peek()` on an empty Queue/Stack throw `InvalidOperationException`. Use `TryDequeue`/`TryPop`/`TryPeek`. Enumerating a Stack yields items in **pop order** (last pushed first).

10.8 LinkedList<T>

✎ Definition

`LinkedList<T>` is a **doubly-linked list** of nodes. It gives O(1) insert and remove at a known node, but O(n) access by position.

A **doubly linked list**: O(1) insertion and removal anywhere *when you already hold the node*, but no index access (O(n) to find).

```
var ll = new LinkedList<int>();
LinkedListNode<int> n1 = ll.AddLast(10);
ll.AddFirst(5);                  // 5 <-> 10
ll.AddAfter(n1, 20);             // 5 <-> 10 <-> 20
ll.AddBefore(n1, 7);             // 5 <-> 7 <-> 10 <-> 20
ll.Remove(n1);                   // O(1) with the node
Console.WriteLine(ll.First!.Value + " " + ll.Last!.Value);   // 5 20
for (var node = ll.First; node != null; node = node.Next) Console.Write(node.Value + " ");
```

★ Interview Tip

Classic use: an **LRU cache** = `Dictionary<key, LinkedListNode>` + `LinkedList` (move a node to the front on access in O(1), evict from the back). In practice `List<T>` is faster for most workloads due to CPU cache locality.

10.9 PriorityQueue<TElement, TPriority> (.NET 6)

✎ Definition

`PriorityQueue<TElement,TPriority>` is a **min-heap**: Dequeue always returns the element with the lowest priority value.

A min-heap: `Dequeue` always returns the element with the **lowest** priority value.

```
var pq = new PriorityQueue<string, int>();
pq.Enqueue("Low task", 5);
pq.Enqueue("Urgent", 1);
pq.Enqueue("Normal", 3);
while (pq.TryDequeue(out var task, out var pri))
    Console.WriteLine($"{pri}: {task}");          // 1 Urgent, 3 Normal, 5 Low task
var maxHeap = new PriorityQueue<string, int>(Comparer<int>.Create((a, b) => b - a));
```

⚡ Tricky Point

Enumerating `pq.UnorderedItems` is **not** in priority order. Also, PriorityQueue does not support updating an item's priority or checking `Contains`.

10.10 Non-Generic Collections (Legacy)

✎ Definition

The pre-generics collections (`ArrayList`, `Hashtable`...) in `System.Collections`. They store `object`, so they are not type-safe and box value types. Avoid them in new code.

```
var al = new ArrayList { 1, "two", 3.0 };    // stores object: boxing + casting, not type-safe
int x = (int)al[0]!;
var ht = new Hashtable { ["a"] = 1 };
object? missing = ht["zzz"];                 // returns null (Dictionary would THROW)
```

ArrayList / Hashtable	List<T> / Dictionary<K,V>
Store `object` - any type mixed	Type-safe at compile time
Boxing for value types; casting needed	No boxing, no casting
Hashtable returns null for a missing key	Dictionary throws KeyNotFoundException
Legacy (.NET 1.0)	Use always in new code

10.11 Concurrent (Thread-Safe) Collections

✎ Definition

Collections in `System.Collections.Concurrent` (`ConcurrentDictionary`, `ConcurrentQueue`, `BlockingCollection`...) that multiple threads can safely use at once without external locks.

Normal collections are **not thread-safe** for writes. Concurrent writes to a `Dictionary` can corrupt it (even causing infinite loops).

```
var cache = new ConcurrentDictionary<string, decimal>();
cache.TryAdd("USD", 83.2m);
decimal rate = cache.GetOrAdd("EUR", key => FetchRate(key));      // atomic get-or-create
cache.AddOrUpdate("USD", 83m, (key, old) => old + 0.1m);         // atomic update
cache.TryRemove("EUR", out _);
cache.TryUpdate("USD", 84m, comparisonValue: 83.3m);             // compare-and-swap

var q = new ConcurrentQueue<int>(); q.Enqueue(1); q.TryDequeue(out var item);
var bag = new ConcurrentBag<int>();   // unordered, optimised for same-thread add/take
var stack = new ConcurrentStack<int>();
// Producer-consumer with BlockingCollection
using var bc = new BlockingCollection<string>(boundedCapacity: 100);
Task.Run(() => { foreach (var f in files) bc.Add(f); bc.CompleteAdding(); });
foreach (var f in bc.GetConsumingEnumerable()) Process(f);   // blocks until items arrive
```

> **⚡ Tricky Point**
>
> `GetOrAdd(key, factory)` is thread-safe for the dictionary, but the **factory delegate may run more than once** for the same key under contention (only one result is stored). If the factory is expensive or has side effects, store `Lazy<T>` values: `GetOrAdd(key, k => new Lazy<T>(() => Create(k))).Value`.

> **i Note**
>
> For modern producer-consumer pipelines, `System.Threading.Channels` (`Channel.CreateBounded<T>`) is the async-friendly alternative to `BlockingCollection`.

10.12 Immutable, ReadOnly and Frozen Collections

> **✎ Definition**
>
> **ReadOnly** collections are read-only *views* over data that can still change underneath. **Immutable** collections can never change. **Frozen** collections (.NET 8) are immutable and optimised for fast reads.

```
var ro = list.AsReadOnly();                // WRAPPER: still changes if 'list' changes!
IReadOnlyList<int> view = list;            // just an interface - can be cast back to List

var im = ImmutableList.Create(1, 2, 3);
var im2 = im.Add(4);                       // returns NEW list; im is unchanged
var builder = im.ToBuilder(); builder.Add(5); var im3 = builder.ToImmutable();  // efficient bulk edits

var frozen = countryCodes.ToFrozenDictionary();   // .NET 8: slow to create, fastest reads
```

Type	Can the caller modify?	Can the source change underneath?
`ReadOnlyCollection<T>`	No	Yes (it's a view)
`IReadOnlyList<T>`	Not directly (cast possible)	Yes
`ImmutableList<T>`	No - operations return new instances	No, never
`FrozenDictionary/FrozenSet`	No	No; optimised for lookup

10.13 Big-O Complexity Cheat Sheet

> ✎ **Definition**
>
> **Big-O notation** describes how the cost of an operation grows with collection size, for example O(1) constant, O(log n) logarithmic and O(n) linear. Use this table to compare collections.

Collection	Access by index	Search / Contains	Insert	Delete	Ordered?
Array `T[]`	O(1)	O(n)	n/a (fixed)	n/a	Insertion order
`List<T>`	O(1)	O(n)	O(1)* end, O(n) middle	O(n)	Insertion order
`LinkedList<T>`	O(n)	O(n)	O(1) with node	O(1) with node	Insertion order
`Dictionary<K, V>`	n/a	O(1) by key	O(1)*	O(1)	No
`HashSet<T>`	n/a	O(1)	O(1)*	O(1)	No
`SortedDictionary<K,V>`	n/a	O(log n)	O(log n)	O(log n)	Sorted by key
`SortedList<K, V>`	O(1) via Keys[i]	O(log n)	O(n)	O(n)	Sorted by key
`SortedSet<T>`	n/a	O(log n)	O(log n)	O(log n)	Sorted
`Queue<T>` / `Stack<T>`	n/a	O(n)	O(1)*	O(1)	FIFO / LIFO
`PriorityQueue<T,P>`	n/a	n/a	O(log n)	O(log n)	By priority

(* amortised: occasionally O(n) when resizing)

10.14 Which Collection Should I Use?

> **✎ Definition**
>
> A decision guide that matches each need (order, uniqueness, key lookup, FIFO/LIFO, priority, thread safety) to the most suitable collection type.

Need	Use
Ordered list, access by index, duplicates OK	`List<T>`
Fixed size, maximum performance	`T[]` array / `Span<T>`
Look up a value by a unique key	`Dictionary<K,V>`
Unique items / fast "exists?" check	`HashSet<T>`
Keys always sorted	`SortedDictionary<K,V>` (or `SortedList` for static data)
Unique and sorted, min/max, ranges	`SortedSet<T>`
Process in arrival order	`Queue<T>`
Undo / back navigation / DFS	`Stack<T>`
Process by priority / scheduling / Dijkstra	`PriorityQueue<T,P>`
Frequent insert/remove in the middle with node references	`LinkedList<T>`
Multiple threads read and write	`ConcurrentDictionary`, `ConcurrentQueue`, `Channel<T>`
Shared data that must never change	`ImmutableList`, `FrozenDictionary`
One key -> many values (read-only)	`ILookup<K,V>` via `ToLookup()`

10.15 Iterators and yield return

> **✎ Definition**
>
> An **iterator** is a method that uses `yield return` to produce a sequence **lazily**, one element at a time. The compiler generates the `IEnumerator` state machine.

`yield return` lets you write a method that produces items **lazily**, one at a time. The compiler generates a state machine implementing `IEnumerable<T>`.

```
IEnumerable<int> Evens(int max)
{
    Console.WriteLine("Started");
    for (int i = 0; i <= max; i += 2)
    {
        yield return i;          // pause here, hand out i, resume on next MoveNext()
    }
    // yield break;              // stop early
}
var e = Evens(10);               // nothing printed yet! (lazy)
foreach (var x in e.Take(2)) Console.WriteLine(x);   // Started, 0, 2

IEnumerable<string> ReadLines(string path)   // stream a 5 GB file using little memory
{
    using var reader = new StreamReader(path);
    string? line;
    while ((line = reader.ReadLine()) != null) yield return line;
}
```

> **⚡ Tricky Point**
>
> Because iterators are lazy, argument validation inside them runs only when enumeration starts, not when the method is called. Wrap the iterator in a normal method that validates and then calls a local iterator function, if you need eager validation. Enumerating the same `IEnumerable` twice runs the code twice.

10.16 Custom Comparers and Equality

> **✎ Definition**
>
> `IComparer<T>` defines custom **ordering**, and `IEqualityComparer<T>` defines custom **equality and hashing**. Sorts, sets and dictionaries use them in place of the default behaviour.

```
// 1) Implement IComparable<T> - natural ordering
public class Employee : IComparable<Employee>
{
    public string Name { get; set; } = ""; public decimal Salary { get; set; }
    public int CompareTo(Employee? other) => Salary.CompareTo(other?.Salary);
}
employees.Sort();   // uses CompareTo

// 2) IComparer<T> - external, multiple orderings
public class ByName : IComparer<Employee>
{
    public int Compare(Employee? x, Employee? y) => string.Compare(x?.Name, y?.Name, StringComparison.Ordinal);
}
employees.Sort(new ByName());
var sorted = new SortedSet<Employee>(new ByName());

// 3) IEqualityComparer<T> - custom equality for Dictionary/HashSet/Distinct
public class RrnComparer : IEqualityComparer<Txn>
{
    public bool Equals(Txn? a, Txn? b) => a?.Rrn == b?.Rrn;
    public int GetHashCode(Txn t) => t.Rrn.GetHashCode();
}
var uniqueTxns = new HashSet<Txn>(txns, new RrnComparer());
```

> **⚡ Tricky Point**
>
> `SortedSet<T>` and `SortedDictionary` treat items where `Compare` returns **0** as duplicates. Sorting employees by salary in a `SortedSet` will silently drop employees with the same salary. Add a tie-breaker (e.g. then by Id).

11. Generics

11.1 Why Generics?

> ✎ **Definition**
>
> **Generics** let you write classes and methods with **type parameters** (<T>), so one piece of code works for any type while staying type-safe and avoiding boxing.

Generics let you write a class or method **once** with a type placeholder `T`, and use it with any type while keeping **compile-time type safety** and **no boxing**.

```
// Without generics: object everywhere - casts and boxing
public class ObjBox { public object? Value; }
var b = new ObjBox { Value = 5 }; int n = (int)b.Value!;   // boxing + cast

// With generics
public class Box<T> { public T Value { get; set; } = default!; }
var ib = new Box<int> { Value = 5 };  int m = ib.Value;      // no cast, no boxing
var sb = new Box<string> { Value = "hi" };
```

11.2 Generic Methods

> ✎ **Definition**
>
> A **generic method** declares its own type parameter (`void Swap<T>(ref T a, ref T b)`). The compiler usually infers T from the arguments.

```
public static void Swap<T>(ref T a, ref T b) => (a, b) = (b, a);
public static T Max<T>(T a, T b) where T : IComparable<T> => a.CompareTo(b) >= 0 ? a : b;

int x = 1, y = 2; Swap(ref x, ref y);      // T inferred as int
Console.WriteLine(Max("apple", "mango"));  // mango
Console.WriteLine(Max<double>(3, 4.5));    // explicit type argument
```

11.3 Generic Constraints

> **✎ Definition**
>
> **Constraints** (`where T : ...`) limit which types can be used for T, such as `class`, `struct`, `new()`, `notnull`, a base class or an interface, so the code can use those members.

Constraint	Meaning
`where T : class`	T must be a reference type
`where T : class?`	Reference type, nullable allowed
`where T : struct`	T must be a non-nullable value type
`where T : notnull`	Non-nullable type (value or reference)
`where T : unmanaged`	Value type with no references (for pointers/Span)
`where T : new()`	Must have a public parameterless constructor (must be last)
`where T : BaseClass`	Must be / derive from BaseClass
`where T : IInterface`	Must implement the interface
`where T : U`	T must derive from another type parameter U
`where T : Enum` / `Delegate`	Enum or delegate types (C# 7.3)
`where T : allows ref struct`	C# 13: T may be a ref struct like Span<T>

```
public class Repository<TEntity, TKey>
    where TEntity : class, IEntity<TKey>, new()
    where TKey : notnull
{
    private readonly Dictionary<TKey, TEntity> _store = new();
    public void Add(TEntity e) => _store[e.Id] = e;
    public TEntity GetOrCreate(TKey id) => _store.TryGetValue(id, out var e) ? e : new TEntity();
}
public interface IEntity<TKey> { TKey Id { get; } }
```

11.4 default(T) and Generic Math

> **✎ Definition**
>
> `default(T)` gives the default value of any type parameter. **Generic math** (.NET 7, static abstract interface members such as `INumber<T>`) allows arithmetic on T.

```
T FirstOrFallback<T>(IList<T> items) => items.Count > 0 ? items[0] : default!;  // null or 0

// Generic math (C# 11 / .NET 7): static abstract interface members
T Sum<T>(IEnumerable<T> values) where T : INumber<T>
{
    T total = T.Zero;
    foreach (var v in values) total += v;
    return total;
}
Sum(new[] { 1, 2, 3 });        // 6
Sum(new[] { 1.5m, 2.5m });     // 4.0
```

11.5 Covariance and Contravariance

> ✎ **Definition**
>
> **Variance** allows implicit conversion of generic interfaces and delegates. **Covariance** (`out T`) uses a more derived type; **contravariance** (`in T`) uses a less derived type.

Variance describes when a generic type with a **more derived** type argument can be used in place of one with a **less derived** argument. Only for **interfaces and delegates**, and only for **reference types**.

```
// Covariance (out T): you only GET T out -> Derived can be used as Base
IEnumerable<string> strings = new List<string> { "a" };
IEnumerable<object> objs = strings;          // OK - IEnumerable<out T>

// Contravariance (in T): you only PUT T in -> Base handler can be used for Derived
Action<object> printAny = o => Console.WriteLine(o);
Action<string> printStr = printAny;          // OK - Action<in T>

// Invariance: List<T> both reads and writes T
// List<object> lo = new List<string>();     // ERROR - otherwise you could add an int to a string list
public interface IProducer<out T> { T Produce(); }
public interface IConsumer<in T> { void Consume(T item); }
```

> ☼ **Simple Analogy**
>
> Covariance: a *basket of mangoes* can be given to someone who asked for *a basket of fruit* - they only take fruit out. Contravariance: a person who can *eat any fruit* can be hired to eat mangoes. But a *fruit box* you can both put into and take out of is invariant: if someone wanting a mango box received a fruit box, they might pull out an apple.

> ⚡ **Tricky Point**
>
> `IEnumerable<int>` is **not** convertible to `IEnumerable<object>` - variance doesn't work with value types (would require boxing each element).

11.6 Static Fields in Generic Types

> ✎ **Definition**
>
> A static field in a generic type is **separate for each closed type**: `Cache<int>.Count` and `Cache<string>.Count` are different variables.

```
public class Counter<T> { public static int Count; }
Counter<int>.Count++;
Counter<string>.Count += 10;
Console.WriteLine(Counter<int>.Count);     // 1 - each closed type has its own static field
```

12. Delegates, Lambdas and Events

12.1 Delegates

> ✎ **Definition**
>
> A **delegate** is a **type-safe reference to a method** (a function pointer) with a specific signature, so methods can be passed around and invoked later.

A delegate is a **type-safe function pointer**: an object that holds a reference to a method with a specific signature. It lets you pass behaviour as a parameter.

```
public delegate decimal FeeCalculator(decimal amount);    // 1. declare delegate type

decimal VisaFee(decimal a) => a * 0.018m;                   // 2. methods with same signature
decimal RupayFee(decimal a) => 0m;

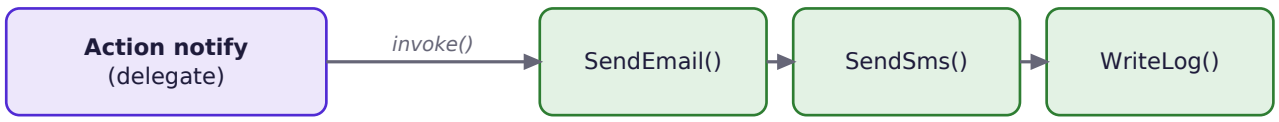
FeeCalculator calc = VisaFee;                               // 3. assign
Console.WriteLine(calc(1000));                              // 18.000 - 4. invoke
calc = RupayFee;
Console.WriteLine(calc.Invoke(1000));                       // 0
decimal ApplyFee(decimal amt, FeeCalculator f) => amt + f(amt);   // pass behaviour
ApplyFee(500, VisaFee);
```

> ☼ **Simple Analogy**
>
> A delegate is like a TV remote's programmable button. The button (delegate) has a fixed shape, and you decide which action (method) it triggers.

12.2 Multicast Delegates

> ✎ **Definition**
>
> A **multicast delegate** holds an invocation list of several methods, added with += and removed with -=, and calls them in order. Only the last method's return value is kept.

A delegate can hold a **list** of methods (invocation list). Invoking it calls all of them in order.



notify += SendEmail; notify += SendSms; notify += WriteLog; → called in order

Figure 12.1 - A multicast delegate calls every method in its invocation list

```
Action notify = () => Console.WriteLine("Email sent");
notify += () => Console.WriteLine("SMS sent");
notify += WriteLog;
notify();            // Email sent, SMS sent, log...
notify -= WriteLog;  // remove

Func<int> f = () => 1;
f += () => 2;
Console.WriteLine(f());   // 2 - only the LAST return value is returned!
```

⚡ Tricky Point

With multicast delegates that return values, you only get the **last** method's result. If one method throws, the remaining methods **do not run**. Use `GetInvocationList()` to call each one individually with your own error handling.

12.3 Built-in Delegates: Func, Action, Predicate

✎ Definition

Ready-made generic delegate types: `Func<..., TResult>` returns a value, `Action<...>` returns void, and `Predicate<T>` returns bool.

Delegate	Signature	Example
`Action`	no params, returns void	`Action a = () => Console.WriteLine("Hi");`
`Action<T1..T16>`	params, returns void	`Action<string> log = m => Console.WriteLine(m);`
`Func<TResult>`	no params, returns TResult	`Func<DateTime> now = () => DateTime.Now;`
`Func<T1..T16, TResult>`	last type argument is the return type	`Func<int, int, int> add = (a, b) => a + b;`
`Predicate<T>`	takes T, returns bool	`Predicate<int> isEven = n => n % 2 == 0;`
`Comparison<T>`	(T, T) -> int	used by `List.Sort`
`EventHandler<TArgs>`	(object? sender, TArgs e)	events

12.4 Anonymous Methods and Lambda Expressions

> **✎ Definition**
>
> Inline, unnamed functions: **anonymous methods** (`delegate(int x) { ... }`) and the shorter **lambda expressions** (`x => x * 2`).

```
Func<int, int> square1 = delegate (int x) { return x * x; };   // anonymous method (C# 2)
Func<int, int> square2 = x => x * x;                           // expression lambda
Func<int, int, int> add = (a, b) => a + b;
Func<string, bool> check = s =>                                // statement lambda
{
    if (string.IsNullOrEmpty(s)) return false;
    return s.Length > 3;
};
Action greet = () => Console.WriteLine("Hello");
var parse = (string s) => int.Parse(s);                        // natural type (C# 10): Func<string,int>
Func<int, int> inc = static x => x + 1;                        // static lambda: cannot capture
var withDefault = (int x = 5) => x * 2;                         // default params (C# 12)
```

12.5 Closures and Captured Variables

> **✎ Definition**
>
> A **closure** is a lambda that **captures variables** from its enclosing scope. It keeps a reference to the variable itself, not a copy of its value, so it extends the variable's lifetime.

A lambda can **capture** local variables from its surrounding scope. The compiler moves captured variables into a hidden class on the heap, so the lambda sees the **variable**, not a snapshot of its value.

```
int counter = 0;
Action incr = () => counter++;
incr(); incr();
Console.WriteLine(counter);     // 2 - the lambda modified the real variable

Func<int> MakeCounter()
{
    int c = 0;
    return () => ++c;           // c lives on after the method returns
}
var next = MakeCounter();
Console.WriteLine(next() + " " + next());   // 1 2
```

The famous loop-capture puzzle

```
var actions = new List<Action>();
for (int i = 0; i < 3; i++)
    actions.Add(() => Console.Write(i + " "));
foreach (var a in actions) a();          // prints: 3 3 3   (all share ONE 'i')
actions.Clear();

for (int i = 0; i < 3; i++)
{
    int copy = i;                        // new variable each iteration
    actions.Add(() => Console.Write(copy + " "));
}                                        // prints: 0 1 2
actions.Clear();

foreach (var i in new[] { 0, 1, 2 })
    actions.Add(() => Console.Write(i)); // C# 5+: foreach gets a fresh variable each time -> 0 1 2
```

> **⚡ Tricky Point**
>
> The `for` loop variable is **one** variable shared by all iterations, so every lambda sees its final value. Since C# 5, `foreach` creates a new variable per iteration, so it behaves "correctly". This is one of the most asked C# output questions.

12.6 Events

> **✎ Definition**
>
> An **event** is a delegate wrapped by the `event` keyword so outsiders can only subscribe (+=) or unsubscribe (-=). It implements the publisher-subscriber pattern.

An event is a **wrapper around a delegate** that restricts outsiders: they can only subscribe (`+=`) or unsubscribe (`-=`). Only the owning class can raise it or reassign it. This is the **Publisher-Subscriber (Observer)** pattern.

```
public class PaymentCompletedEventArgs : EventArgs
{
    public string TxnId { get; init; } = "";
    public decimal Amount { get; init; }
}

public class PaymentService                                    // PUBLISHER
{
    public event EventHandler<PaymentCompletedEventArgs>? PaymentCompleted;
    public void Pay(string id, decimal amt)
    {
        // ... process payment ...
        OnPaymentCompleted(new PaymentCompletedEventArgs { TxnId = id, Amount = amt });
    }
    protected virtual void OnPaymentCompleted(PaymentCompletedEventArgs e)
        => PaymentCompleted?.Invoke(this, e);                  // thread-safe null check
}

var svc = new PaymentService();                                // SUBSCRIBERS
svc.PaymentCompleted += (sender, e) => Console.WriteLine($"Email for {e.TxnId}");
svc.PaymentCompleted += OnPaid;
svc.Pay("T100", 2500);
void OnPaid(object? s, PaymentCompletedEventArgs e) => Console.WriteLine($"Ledger +{e.Amount}");
```

delegate field	event
Outsiders can invoke it: `obj.Handler()`	Only the declaring class can raise it
Outsiders can overwrite with `=` (wipes other subscribers)	Outsiders can only use `+=` and `-=`
General callback mechanism	Notification mechanism (observer pattern)

> **⚠ Watch Out**
>
> **Memory leak:** if a long-lived publisher (e.g. a static service) holds an event subscription to a short-lived subscriber (e.g. a form/view model), the subscriber can never be garbage-collected. Always unsubscribe (`-=`) when done, typically in `Dispose`. Lambdas can't be unsubscribed unless you keep a reference to them.

12.7 Expression Trees

> **✎ Definition**
>
> An **expression tree** (`Expression<Func<...>>`) represents code as a **data structure** that can be inspected and translated, for example into SQL by Entity Framework.

`Func<T>` is compiled code. `Expression<Func<T>>` is **data describing code** that can be inspected and translated - this is how Entity Framework turns LINQ into SQL.

```
Func<int, bool> f = x => x > 5;                          // executable delegate
Expression<Func<int, bool>> e = x => x > 5;              // tree: Lambda(GreaterThan(Param x, Const 5))
Console.WriteLine(e.Body);                               // (x > 5)
var compiled = e.Compile();                              // turn into a delegate
```

13. LINQ (Language Integrated Query)

13.1 What is LINQ?

> ✎ **Definition**
>
> **LINQ** (Language Integrated Query) is a set of query operators and syntax for filtering, projecting, sorting and grouping data from any source (collections, databases, XML) directly in C#.

LINQ lets you query **any data source** (collections, databases, XML, JSON) with a consistent C# syntax. It is a set of **extension methods** on `IEnumerable<T>` (LINQ to Objects) and `IQueryable<T>` (LINQ to Entities/SQL), living in `System.Linq`.

```
var txns = new List<Txn>
{
    new("T1", 500m, "SUCCESS", "VISA"),
    new("T2", 1500m, "FAILED", "RUPAY"),
    new("T3", 2500m, "SUCCESS", "VISA"),
    new("T4", 300m, "SUCCESS", "MASTER"),
};
public record Txn(string Id, decimal Amount, string Status, string Network);

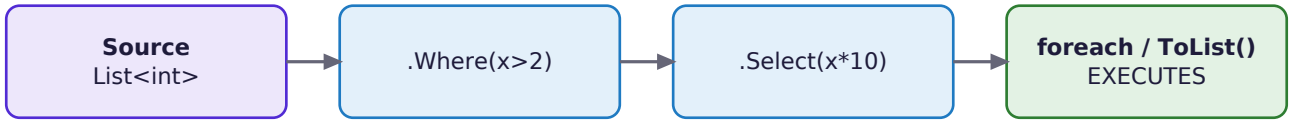
// Query syntax (SQL-like)
var q1 = from t in txns
         where t.Status == "SUCCESS" && t.Amount > 400
         orderby t.Amount descending
         select new { t.Id, t.Amount };

// Method syntax (fluent, more powerful) - same result
var q2 = txns.Where(t => t.Status == "SUCCESS" && t.Amount > 400)
             .OrderByDescending(t => t.Amount)
             .Select(t => new { t.Id, t.Amount });
```

> i **Note**
>
> The compiler translates query syntax into method calls. Some operators (`Count`, `Distinct`, `Take`, `First`, `ToList`...) exist only in method syntax. Query syntax shines for **joins** and `let`.

13.2 Deferred vs Immediate Execution

> ✎ **Definition**
>
> **Deferred** execution means a query runs only when it is enumerated, and runs again each time. **Immediate** execution (`ToList`, `Count`, `First`...) runs it at once and stores the result.



Nothing runs until you enumerate (deferred execution). Each item flows through the whole pipeline one by one.

Figure 13.1 - A LINQ query is a pipeline that runs only when enumerated

```
var nums = new List<int> { 1, 2, 3 };
var query = nums.Where(n => n > 1);     // NOT executed yet - just a recipe
nums.Add(4);
Console.WriteLine(string.Join(",", query));   // 2,3,4  <- sees the added item!

var snapshot = nums.Where(n => n > 1).ToList();  // executed NOW (immediate)
nums.Add(5);
Console.WriteLine(snapshot.Count);       // 3 - not affected
```

Deferred (lazy) - return IEnumerable	Immediate - force execution
`Where, Select, SelectMany, OrderBy, ThenBy, GroupBy, Join, Skip, Take, Distinct, Concat, Zip, Reverse, Cast, OfType`	`ToList, ToArray, ToDictionary, ToHashSet, ToLookup, Count, Sum, Min, Max, Average, First, Single, Last, Any, All, Contains, Aggregate`

> ⚡ **Tricky Point**
>
> **Multiple enumeration trap:** each `foreach`/`Count()` over a deferred query re-runs the whole pipeline. If the source is a DB query or an expensive method, you hit the database or recompute every time. Call `.ToList()` once if you need the results more than once. (Tools like ReSharper warn "Possible multiple enumeration".)

13.3 Filtering and Projection

> ✎ **Definition**
>
> **Filtering** (`Where`, `OfType`) selects the elements that match a condition. **Projection** (`Select`, `SelectMany`) transforms each element into a new shape.

```
var visa = txns.Where(t => t.Network == "VISA");
var withIndex = txns.Where((t, i) => i % 2 == 0);          // index overload
var ids = txns.Select(t => t.Id);                          // projection
var dto = txns.Select((t, i) => new { Row = i + 1, t.Id });
var strings = mixedList.OfType<string>();                  // filter by type
var asObj = list.Cast<object>();                           // cast each (throws on failure)

// SelectMany - flatten nested collections
var orders = new[] { new { Id = 1, Items = new[] { "Pen", "Ink" } },
                     new { Id = 2, Items = new[] { "Book" } } };
var allItems = orders.SelectMany(o => o.Items);            // Pen, Ink, Book
var pairs = orders.SelectMany(o => o.Items, (o, item) => $"{o.Id}:{item}");  // 1:Pen 1:Ink 2:Book
```

★ Interview Tip

`Select` gives one output per input (a list of lists stays nested). `SelectMany` flattens: one input can produce many outputs merged into a single sequence.

13.4 Element Operators: First, Single, Last, ElementAt

✎ Definition

Operators that return a **single element**: `First`, `Single`, `Last`, `ElementAt`. They differ in how they treat zero or multiple matches; the `...OrDefault` versions return a default value instead of throwing.

```
var t1 = txns.First(t => t.Amount > 1000);        // T2; throws if none
var t2 = txns.FirstOrDefault(t => t.Amount > 9999); // null if none
var t3 = txns.FirstOrDefault(t => t.Amount > 9999, defaultValue: Txn.Empty); // .NET 6
var one = txns.Single(t => t.Id == "T3");         // throws if 0 OR more than 1
var maybe = txns.SingleOrDefault(t => t.Id == "X"); // null if 0; throws if > 1!
var last = txns.Last();
var third = txns.ElementAt(2);
var safe = txns.ElementAtOrDefault(10);
```

Method	0 matches	1 match	2+ matches
`First()`	Exception	Returns it	Returns first
`FirstOrDefault()`	default/null	Returns it	Returns first
`Single()`	Exception	Returns it	Exception
`SingleOrDefault()`	default/null	Returns it	**Exception**

⚡ Tricky Point

`FirstOrDefault()` on `List<int>` returns **0** when empty - ambiguous. And `SingleOrDefault` still throws when there are multiple matches. Use `Single` when exactly one is a business rule (e.g. lookup by primary key) - it catches data bugs early.

13.5 Ordering

✎ Definition

Sorting a sequence with `OrderBy`/`OrderByDescending`, then adding secondary keys with ThenBy/ThenByDescending. LINQ ordering is stable.

```
var byAmt = txns.OrderBy(t => t.Amount);
var multi = txns.OrderBy(t => t.Network).ThenByDescending(t => t.Amount);
var rev = txns.AsEnumerable().Reverse();
var sortedNums = new[] { 3, 1, 2 }.Order();               // .NET 7
// WRONG: txns.OrderBy(a).OrderBy(b) - second OrderBy replaces the first. Use ThenBy.
```

13.6 Aggregation

✎ Definition

Operators that **reduce a sequence to one value**: `Count`, `Sum`, `Min`, `Max`, `Average`, `MinBy`/`MaxBy` and the general-purpose `Aggregate`.

```
int count = txns.Count(t => t.Status == "SUCCESS");   // 3
decimal total = txns.Sum(t => t.Amount);              // 4800
decimal avg = txns.Average(t => t.Amount);
decimal max = txns.Max(t => t.Amount);
Txn? biggest = txns.MaxBy(t => t.Amount);             // .NET 6: returns the element
Txn? smallest = txns.MinBy(t => t.Amount);
long big = txns.LongCount();

int product = new[] { 1, 2, 3, 4 }.Aggregate((acc, x) => acc * x);   // 24
string csv = new[] { "a", "b", "c" }.Aggregate((a, b) => a + "," + b); // a,b,c
decimal net = txns.Aggregate(0m, (acc, t) => acc + (t.Status == "SUCCESS" ? t.Amount : 0));
```

⚡ Tricky Point

`Max()`, `Min()`, `Average()` on an **empty** sequence of non-nullable numbers throw `InvalidOperationException`; `Sum()` returns 0. Use `DefaultIfEmpty()` or nullable selectors (`Max(t => (decimal?)t.Amount)`) to get null instead.

13.7 Quantifiers

✎ Definition

Operators that return a **bool** about a sequence: `Any` (at least one matches), `All` (every element matches) and `Contains` (a value is present).

```
bool anyFailed = txns.Any(t => t.Status == "FAILED");   // true; stops at first match
bool hasItems = txns.Any();                              // prefer over Count() > 0
bool allPositive = txns.All(t => t.Amount > 0);
bool contains = ids.Contains("T2");
bool emptyAll = new List<int>().All(x => x > 100);       // TRUE! (vacuous truth)
```

13.8 Grouping

✎ Definition

`GroupBy` splits a sequence into **groups that share a key**; each group is an `IGrouping<TKey,T>`. `ToLookup` does the same eagerly.

```
var byNetwork = txns.GroupBy(t => t.Network);
foreach (var g in byNetwork)
{
    Console.WriteLine($"{g.Key}: count={g.Count()} total={g.Sum(t => t.Amount)}");
}
// VISA: count=2 total=3000, RUPAY: count=1 total=1500, MASTER: count=1 total=300

var summary = txns.GroupBy(t => new { t.Network, t.Status })     // composite key
                  .Select(g => new { g.Key.Network, g.Key.Status, Total = g.Sum(x => x.Amount) });
```

```
var q = from t in txns
        group t by t.Status into g
        where g.Count() > 1
        select new { Status = g.Key, Count = g.Count() };

var counts = txns.CountBy(t => t.Network);          // .NET 9: KeyValuePair<string,int>
var lookup = txns.ToLookup(t => t.Network);         // immediate, like a read-only multi-dictionary
var visaTxns = lookup["VISA"];                      // never throws; empty if key missing
```

13.9 Joins

✎ Definition

Joins combine two sequences by matching keys: `Join` (inner join), `GroupJoin` (for left outer joins) and `Zip` (pairs elements by position).

```
var networks = new[] { new { Code = "VISA", Name = "Visa Inc" },
                       new { Code = "RUPAY", Name = "NPCI RuPay" } };

// INNER JOIN
var inner = txns.Join(networks,
                      t => t.Network, n => n.Code,
                      (t, n) => new { t.Id, n.Name });
var innerQ = from t in txns
             join n in networks on t.Network equals n.Code
             select new { t.Id, n.Name };
// LEFT OUTER JOIN (classic pattern)
var left = from t in txns
           join n in networks on t.Network equals n.Code into gj
           from n in gj.DefaultIfEmpty()
           select new { t.Id, Name = n?.Name ?? "(unknown)" };      // T4 MASTER -> (unknown)
var left9 = txns.LeftJoin(networks, t => t.Network, n => n.Code,
                          (t, n) => new { t.Id, Name = n?.Name });  // .NET 10 LeftJoin

// GROUP JOIN (one-to-many)
var perNetwork = networks.GroupJoin(txns, n => n.Code, t => t.Network,
                         (n, ts) => new { n.Name, Count = ts.Count() });

// Reconciliation style: items in switch file but missing in bank file
var missingInBank = switchTxns.ExceptBy(bankTxns.Select(b => b.Rrn), s => s.Rrn);   // .NET 6
```

13.10 Set Operators

✎ Definition

Operators that treat sequences as **mathematical sets**: `Distinct`, `Union`, `Intersect`, `Except`, plus the key-based `...By` variants.

```
int[] a = { 1, 2, 2, 3 }, b = { 3, 4 };
a.Distinct();        // 1,2,3
a.Union(b);          // 1,2,3,4 (distinct)
a.Concat(b);         // 1,2,2,3,3,4 (keeps duplicates)
a.Intersect(b);      // 3
a.Except(b);         // 1,2
txns.DistinctBy(t => t.Network);        // .NET 6: first txn per network
txns.UnionBy(other, t => t.Id); txns.IntersectBy(keys, t => t.Id);
```

> ⚡ **Tricky Point**
>
> `Distinct()`, `Union()`, etc. on **classes** use reference equality by default - two objects with the same data are both kept. Use records, override `Equals`/`GetHashCode`, pass an `IEqualityComparer<T>`, or use `DistinctBy`.

13.11 Partitioning, Paging and Others

> ✎ **Definition**
>
> **Partitioning** operators take or skip part of a sequence (`Take`, `Skip`, `TakeWhile`, `Chunk`). They are the building blocks for **paging** results.

```
var page = txns.Skip((pageNo - 1) * pageSize).Take(pageSize);   // pagination
var firstBig = txns.TakeWhile(t => t.Amount < 2000);   // stops at first failure
var rest = txns.SkipWhile(t => t.Amount < 2000);
var lastTwo = txns.TakeLast(2);
var range = txns.Take(1..3);                           // .NET 6 range support
foreach (var batch in txns.Chunk(1000)) BulkInsert(batch);   // .NET 6: arrays of 1000
var zipped = names.Zip(ages, (n, a) => $"{n}:{a}");
var numbers = Enumerable.Range(1, 5);                  // 1..5
var zeros = Enumerable.Repeat(0, 3);                   // 0,0,0
var none = Enumerable.Empty<int>();
var dict = txns.ToDictionary(t => t.Id, t => t.Amount);   // throws on duplicate keys!
var indexed = txns.Index();                            // .NET 9: (Index, Item) pairs
var sumBy = txns.AggregateBy(t => t.Network, 0m, (acc, t) => acc + t.Amount);  // .NET 9
```

> ⚡ **Tricky Point**
>
> `ToDictionary` throws `ArgumentException` on a duplicate key - common with dirty reconciliation data. Use `GroupBy(...).ToDictionary(g => g.Key, g => g.First())` or `ToLookup`.

13.12 IEnumerable vs IQueryable

> ✎ **Definition**
>
> `IEnumerable<T>` runs queries **in memory** using delegates. `IQueryable<T>` builds expression trees that a provider translates and runs **at the data source**, such as SQL.

IEnumerable<T>	IQueryable<T>
LINQ to Objects - runs in memory (C# delegates)	LINQ to Entities/SQL - builds an expression tree
Filters after data is loaded into memory	Translated to SQL and filtered in the database
`System.Linq.Enumerable` methods take `Func<>`	`System.Linq.Queryable` methods take `Expression<Func<>>`
Good for in-memory collections	Good for remote data sources (EF Core, OData)

```
IQueryable<Order> q = db.Orders.Where(o => o.Total > 1000);   // SQL: WHERE Total > 1000
var list = q.ToList();                                         // query executes here

IEnumerable<Order> e = db.Orders;                              // switched to IEnumerable!
var bad = e.Where(o => o.Total > 1000).ToList();               // SELECT * FROM Orders, filter in memory!
```

> **⚠ Watch Out**
>
> Calling a custom C# method inside an EF Core `Where` (e.g. `Where(o => IsVip(o))`) cannot be translated to SQL and throws at runtime. Also, calling `.AsEnumerable()`/`.ToList()` too early pulls the whole table into memory.

13.13 PLINQ (Parallel LINQ)

> **✎ Definition**
>
> **PLINQ** (`AsParallel()`) runs LINQ queries in **parallel** across multiple CPU cores to speed up CPU-bound work on large data sets.

```
var results = bigList.AsParallel()
                     .WithDegreeOfParallelism(4)
                     .Where(x => ExpensiveCheck(x))
                     .AsOrdered()           // keep source order (costs performance)
                     .ToList();
```

Useful for **CPU-bound** work on large data sets. Not useful (even harmful) for small collections or I/O-bound work.

14. Multithreading, Tasks and Async/Await

14.1 Key Concepts

> ✎ **Definition**
>
> Core concurrency terms: a **process** is a running program, a **thread** is a unit of execution within it, **concurrency** means making progress on several tasks, and **parallelism** means running them at literally the same time.

Term	Meaning
Process	A running program with its own memory space
Thread	A unit of execution inside a process; threads share the heap
Concurrency	Multiple tasks making progress in overlapping time (not necessarily simultaneously)
Parallelism	Multiple tasks running at literally the same time on different CPU cores
Asynchrony	Not blocking a thread while waiting (I/O: DB, HTTP, files)
CPU-bound work	Heavy computation - use `Task.Run`/`Parallel`
I/O-bound work	Waiting on external resources - use `async`/`await` (no extra thread needed)

> ☼ **Simple Analogy**
>
> A restaurant: **synchronous** = a waiter takes an order and stands at the kitchen until the food is ready. **Asynchronous** = the waiter gives the order to the kitchen and serves other tables; when the food is ready, any available waiter delivers it. **Parallel** = several cooks chopping vegetables at the same time.

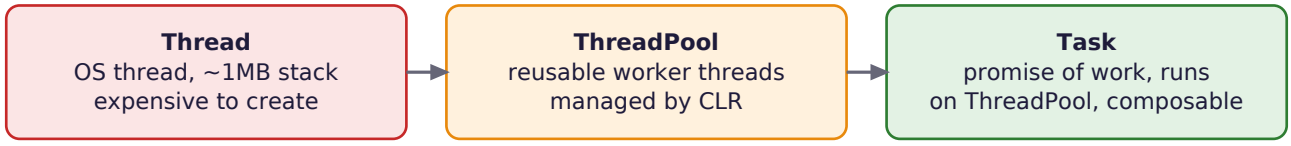


Figure 14.1 - Thread, ThreadPool and Task

14.2 Threads

> **✎ Definition**
>
> A Thread is an OS-level **path of execution** you create and control directly. It is expensive to create, so modern code prefers tasks and the thread pool.

```
var t = new Thread(() =>
{
    Console.WriteLine($"Worker thread {Environment.CurrentManagedThreadId}");
    Thread.Sleep(1000);           // blocks THIS thread
});
t.IsBackground = true;            // background threads don't keep the process alive
t.Start();
t.Join();                         // wait for it to finish
```

> **i Note**
>
> Creating threads manually is rarely needed today. Each thread costs about 1 MB of stack and kernel resources. Use `Task` and the ThreadPool instead.

14.3 Tasks

> **✎ Definition**
>
> A Task (or Task<T>) represents an **asynchronous operation** that will finish in the future. It usually runs on the thread pool and can be awaited, composed and cancelled.

A `Task` represents an asynchronous operation that will complete in the future (a "promise"). `Task<T>` also carries a result.

```
Task<int> t1 = Task.Run(() => HeavyCalculation());   // CPU work on a ThreadPool thread
int result = await t1;

Task done = Task.CompletedTask;
Task<int> ready = Task.FromResult(42);
Task delay = Task.Delay(1000);                        // non-blocking wait

// Run several in parallel and wait for all
Task<string> a = DownloadAsync("url1");
Task<string> b = DownloadAsync("url2");
string[] both = await Task.WhenAll(a, b);             // total time = slowest, not sum
Task<string> first = await Task.WhenAny(a, b);        // whichever finishes first
```

14.4 async and await

> ✎ **Definition**
>
> async marks a method that can await. await pauses the method **without blocking the thread** until a task completes, then resumes. The compiler builds a state machine.

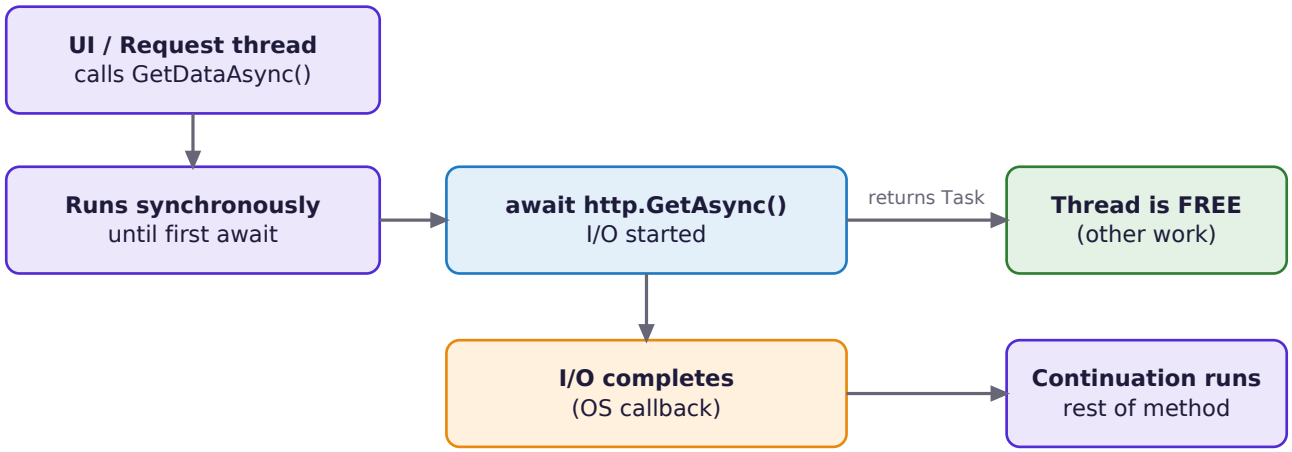


Figure 14.2 - What happens at an await

```
public async Task<decimal> GetBalanceAsync(string accNo)
{
    Console.WriteLine("Before await");              // runs synchronously on caller's thread
    var json = await _http.GetStringAsync($"/accounts/{accNo}");   // thread released here
    Console.WriteLine("After await");                // continuation (maybe another thread)
    return JsonSerializer.Deserialize<Account>(json)!.Balance;
}
```

How it works:

- `async` lets you use `await` in the method and makes the compiler build a **state machine**.
- At `await`, if the task isn't complete, the method **returns** an incomplete Task to its caller and the thread is free.
- When the awaited operation completes, the rest of the method (continuation) is scheduled - on the captured **SynchronizationContext** (UI thread) if there is one, otherwise on a

  ThreadPool thread.
- `async` **does not create a new thread**. For I/O, no thread is used during the wait at all.

Return type	Use
`Task`	Async method with no result
`Task<T>`	Async method returning T
`ValueTask` / `ValueTask<T>`	Hot paths that often complete synchronously (avoid allocation); await only once
`void`	ONLY for event handlers
`IAsyncEnumerable<T>`	Async streams: `await foreach`

14.5 Sequential vs Concurrent Awaits

> ✎ **Definition**
>
> **Sequential** awaits run tasks one after another (total time = sum). **Concurrent** awaits start all tasks first and then `await Task.WhenAll` (total time = the slowest task).

```
// SLOW: 3 seconds total - each waits for the previous
var u = await GetUserAsync();      // 1s
var o = await GetOrdersAsync();    // 1s
var p = await GetPaymentsAsync();  // 1s

// FAST: ~1 second total - start all, then await
var uTask = GetUserAsync();
var oTask = GetOrdersAsync();
var pTask = GetPaymentsAsync();
await Task.WhenAll(uTask, oTask, pTask);
var user = uTask.Result;           // safe here - already completed
```

> ⚠ **Watch Out**
>
> EF Core `DbContext` is **not thread-safe**: running two queries concurrently on the same context with `Task.WhenAll` throws `InvalidOperationException`. Use separate contexts (`IDbContextFactory`) for concurrent queries.

14.6 Async Pitfalls and Tricky Points

> ✎ **Definition**
>
> Common async mistakes: deadlocks from `.Result`/`.Wait()`, `async void`, forgetting to `await`, misusing `Task.Run`, and eliding `async`/`await` unsafely.

1. Deadlock with .Result / .Wait()

```
// In ASP.NET (classic) / WinForms / WPF:
public string GetData() => GetDataAsync().Result;   // DEADLOCK!
// The UI thread blocks waiting for the task; the task's continuation needs the UI thread.
```

Fix: **"async all the way"** - make the caller async and `await`. Library code can use `ConfigureAwait(false)` so continuations don't need the original context. (ASP.NET Core has no SynchronizationContext, so it doesn't deadlock this way, but blocking still wastes threads and causes thread-pool starvation.)

2. async void

```
async void Bad() { throw new Exception(); }   // exception cannot be caught by caller - crashes the process
async Task Good() { throw new Exception(); }  // exception stored in Task, rethrown on await
```

3. Forgetting to await

```
SaveAsync();          // fire-and-forget: exceptions lost, may run after the request ends
await SaveAsync();    // correct
```

4. Task.Run in ASP.NET Core

Wrapping I/O in `Task.Run` just moves the wait to another thread-pool thread - it doesn't help scalability. Use true async APIs (`ReadAsync`, `ExecuteReaderAsync`).

5. Elision of async/await

```
public Task<string> GetAsync() => _http.GetStringAsync(url);  // fine: no state machine
// BUT inside 'using' or try/catch you must await, otherwise the object is disposed
// before the task finishes, and exceptions escape the try block.
```

> **★ Interview Tip**
>
> "Does async/await make code run faster?" - Not by itself. It improves **scalability and responsiveness**: threads aren't blocked while waiting on I/O, so a server can handle more requests with fewer threads and a UI stays responsive.

14.7 Cancellation

> **✎ Definition**
>
> **Cooperative cancellation**: a `CancellationTokenSource` signals a `CancellationToken` that long-running or async operations check so they can stop early.

```
public async Task ProcessFilesAsync(IEnumerable<string> files, CancellationToken ct)
{
    foreach (var f in files)
    {
        ct.ThrowIfCancellationRequested();                // cooperative check
        await ProcessAsync(f, ct);                        // pass token down
    }
}
using var cts = new CancellationTokenSource(TimeSpan.FromSeconds(30));   // auto-cancel after 30s
try { await ProcessFilesAsync(files, cts.Token); }
catch (OperationCanceledException) { Console.WriteLine("Cancelled"); }
cts.Cancel();   // manual cancel
```

14.8 Thread Synchronisation

> **✎ Definition**
>
> Techniques that stop threads from corrupting shared data: `lock`/`Monitor`, `Interlocked`, `SemaphoreSlim`, `Mutex`, `ReaderWriterLockSlim` and the .NET 9 `Lock` type.

When multiple threads update shared data, you get **race conditions**.

```
int counter = 0;
Parallel.For(0, 100_000, _ => counter++);    // result is LESS than 100000! (read-modify-write race)

// 1) lock (Monitor) - mutual exclusion
private readonly object _sync = new();
lock (_sync) { counter++; }

// 2) Interlocked - atomic operations, fastest for simple counters
Interlocked.Increment(ref counter);
Interlocked.Add(ref total, 5);
Interlocked.CompareExchange(ref value, newVal, expected);

// 3) SemaphoreSlim - limit concurrency, async-friendly (lock can't contain await)
private readonly SemaphoreSlim _gate = new(3);       // max 3 at a time
await _gate.WaitAsync();
try { await CallBankApiAsync(); } finally { _gate.Release(); }

// 4) .NET 9 Lock type
private readonly Lock _lock = new();
lock (_lock) { /* ... */ }
```

Primitive	Use for
`lock`/`Monitor`	Protect a short critical section (no await inside)
`Interlocked`	Atomic increment/exchange of numbers/references
`SemaphoreSlim`	Throttling, async mutual exclusion
`ReaderWriterLockSlim`	Many readers, few writers
`Mutex`	Cross-process locking (slow)
`ManualResetEventSlim` / `CountdownEvent` / `Barrier`	Signalling between threads
`volatile`	Prevent caching/reordering of a single field (not atomicity)

⚡ Tricky Point

`await` is **not allowed inside a lock** block (compile error CS1996) because the continuation may run on a different thread that doesn't own the lock. Use `SemaphoreSlim(1,1)` instead. Never `lock(this)` or `lock("string")` - outside code can lock the same object, causing deadlocks.

Deadlock example

```
// Thread 1: lock(A) then lock(B)     Thread 2: lock(B) then lock(A)   -> both wait forever
// Fix: always acquire locks in the same global order; keep critical sections small; use timeouts.
```

14.9 Parallel Class

✎ Definition

The `Parallel` class (`Parallel.For`, `ForEach`, `ForEachAsync`, `Invoke`) splits **CPU-bound work across cores** automatically using the thread pool.

```
Parallel.For(0, 100, i => Compute(i));
Parallel.ForEach(files, new ParallelOptions { MaxDegreeOfParallelism = 4 }, f => Parse(f));
await Parallel.ForEachAsync(urls, new ParallelOptions { MaxDegreeOfParallelism = 5 },
    async (url, ct) => await DownloadAsync(url, ct));     // .NET 6 - async-friendly
Parallel.Invoke(() => TaskA(), () => TaskB());
```

14.10 Async Streams and Channels

✎ Definition

Async streams (`IAsyncEnumerable<T>` with `await foreach`) produce data asynchronously, item by item. **Channels** are thread-safe async queues for producer/consumer pipelines.

```
public async IAsyncEnumerable<Txn> ReadTxnsAsync([EnumeratorCancellation] CancellationToken ct = default)
{
    await foreach (var line in File.ReadLinesAsync("txns.csv", ct))   // .NET 7
        yield return Parse(line);
}
await foreach (var t in ReadTxnsAsync()) Console.WriteLine(t.Id);

var channel = Channel.CreateBounded<string>(100);
_ = Task.Run(async () => { await channel.Writer.WriteAsync("job1"); channel.Writer.Complete(); });
await foreach (var job in channel.Reader.ReadAllAsync()) Console.WriteLine(job);
```

14.11 Thread-Safe Lazy Initialisation and Singleton

✎ Definition

Creating an object **once, on first use, safely across threads**, for example with Lazy<T> or a static initialiser. This is the standard way to build a thread-safe Singleton.

```
public sealed class ConfigCache
{
    private static readonly Lazy<ConfigCache> _instance = new(() => new ConfigCache());  // thread-safe by default
    public static ConfigCache Instance => _instance.Value;
    private ConfigCache() { }
}
[ThreadStatic] private static int _perThread;          // separate value per thread
private static AsyncLocal<string> _correlationId = new();  // flows with async calls
```

15. Advanced C# Features

15.1 Extension Methods

✎ Definition

An **extension method** is a static method in a static class whose first parameter uses `this`. It can then be called as if it were an instance method of that type, which you don't have to own.

Add methods to existing types **without modifying or inheriting** them. Must be a `static` method in a non-nested, non-generic `static` class, with `this` before the first parameter.

```
public static class StringExtensions
{
    public static bool IsNumeric(this string s) => s.All(char.IsDigit);
    public static string Mask(this string card, int visible = 4) =>
        new string('*', card.Length - visible) + card[^visible..];
}
Console.WriteLine("4111111111111111".Mask());   // ************1111
Console.WriteLine("12345".IsNumeric());         // true
// Compiler rewrites to: StringExtensions.Mask("4111...")
```

⚡ Tricky Point

If the type already has an **instance method** with the same signature, the instance method always wins and your extension is silently ignored. Extension methods can be called on a **null** reference (no NullReferenceException at the call - `this` is just a parameter). They cannot access private members. C# 14 adds *extension members* (`extension(string s) { ... }` blocks) which also allow extension properties.

15.2 Nullable Reference Types (C# 8+)

✎ Definition

A compiler feature (`#nullable enable`) where reference types are **non-nullable by default** and `string?` marks the ones that may be null. The compiler then warns about possible null dereferences.

With `<Nullable>enable</Nullable>`, reference types are **non-nullable by default** and the compiler warns about possible null dereferences.

```
string name = null;          // warning CS8600
string? middle = null;       // OK - explicitly nullable
int len = middle.Length;     // warning CS8602: possible null dereference
int len2 = middle?.Length ?? 0;
int len3 = middle!.Length;   // '!' null-forgiving: "trust me" - silences the warning only

public string GetName([NotNullWhen(true)] out string? n) { ... }   // attributes guide analysis
```

⚡ Tricky Point

Nullable reference types are **compile-time warnings only**. At runtime nothing changes - a `string` can still hold null (from JSON, reflection, old libraries). Still validate public inputs.

15.3 Attributes

> **✎ Definition**
>
> An **attribute** is declarative **metadata** attached to code elements with `[Name]`. Compilers, frameworks or reflection read it at compile time or run time.

Attributes attach **metadata** to code, read later by the compiler, frameworks or reflection.

```
[Obsolete("Use ProcessV2", error: false)]
public void Process() { }

[Serializable] public class Settings { }

public class TxnDto
{
    [Required, StringLength(12)] public string Rrn { get; set; } = "";
    [JsonPropertyName("amt")] public decimal Amount { get; set; }
    [JsonIgnore] public string? Internal { get; set; }
}

// Custom attribute
[AttributeUsage(AttributeTargets.Property)]
public class MaskAttribute : Attribute
{
    public int Visible { get; }
    public MaskAttribute(int visible = 4) => Visible = visible;
}
```

15.4 Reflection

> **✎ Definition**
>
> **Reflection** (`System.Reflection`) lets a program **inspect and use type metadata at run time**: list members, create instances, invoke methods and read attributes dynamically.

Reflection inspects types and members **at runtime** using metadata.

```
Type t = typeof(TxnDto);                          // or obj.GetType()
foreach (PropertyInfo p in t.GetProperties())
{
    var mask = p.GetCustomAttribute<MaskAttribute>();
    Console.WriteLine($"{p.Name} : {p.PropertyType.Name} masked={mask != null}");
}
object? instance = Activator.CreateInstance(t);
t.GetProperty("Rrn")!.SetValue(instance, "123456789012");
MethodInfo? m = t.GetMethod("ToString");
var result = m!.Invoke(instance, null);
var priv = t.GetField("_secret", BindingFlags.NonPublic | BindingFlags.Instance);
```

> **i Note**
>
> Reflection is powerful (serializers, DI containers, ORMs, test frameworks) but **slow** and breaks with trimming/Native AOT. Modern .NET favours **source generators** that generate the code at compile time instead.

15.5 typeof vs GetType vs is

✎ Definition

`typeof(T)` gets a Type at **compile time** from a type name. `obj.GetType()` gets the **actual run-time** type of an instance. `is` checks compatibility, including inheritance.

```
Animal a = new Dog();
a.GetType() == typeof(Animal)   // false - runtime type is Dog
a.GetType() == typeof(Dog)      // true
a is Animal                     // true - 'is' respects inheritance
typeof(Animal).IsAssignableFrom(a.GetType())   // true
```

15.6 dynamic

✎ Definition

`dynamic` **bypasses compile-time type checking**. Member access is resolved at run time by the DLR, which gives flexibility (COM, JSON, interop) at the cost of safety and speed.

```
dynamic d = 10;
d = d + 5;            // resolved at runtime
d = "text";           // type can change
d.NonExisting();      // compiles! RuntimeBinderException at runtime
dynamic json = JsonConvert.DeserializeObject("{ 'a': 1 }")!;  // handy with COM/JSON/Python interop
```

15.7 Unsafe Code and Pointers

✎ Definition

`unsafe` code allows **pointers** and direct memory manipulation (`*`, `&`, `fixed`, `stackalloc`) outside the CLR's memory-safety checks. It requires the `AllowUnsafeBlocks` setting.

```
unsafe   // needs <AllowUnsafeBlocks>true</AllowUnsafeBlocks>
{
    int x = 10;
    int* p = &x;
    *p = 20;                        // x is now 20
    fixed (byte* bp = buffer) { }   // pin managed array so GC won't move it
}
```

15.8 File I/O

✎ Definition

Reading and writing files and directories through `System.IO`: `File`, `Directory`, `Path`, streams, `StreamReader`/`StreamWriter` and their async versions.

```
File.WriteAllText("a.txt", "hello");
string all = File.ReadAllText("a.txt");
string[] lines = File.ReadAllLines("a.txt");            // loads whole file
foreach (var l in File.ReadLines("big.csv")) { }        // lazy, streams line by line
File.AppendAllLines("log.txt", new[] { "entry" });
await File.WriteAllTextAsync("b.txt", "async");
bool exists = File.Exists("a.txt");
string path = Path.Combine(baseDir, "reports", "jan.csv");   // never concatenate paths manually
string ext = Path.GetExtension(path);                   // ".csv"
Directory.CreateDirectory("out");
var files = Directory.GetFiles("in", "*.xlsx", SearchOption.AllDirectories);

using var reader = new StreamReader("big.csv");
while (await reader.ReadLineAsync() is { } line) Process(line);
```

15.9 JSON Serialization (System.Text.Json)

✎ Definition

Serialization converts objects to JSON text, and deserialization converts it back. `System.Text.Json` (`JsonSerializer`) is the fast, built-in .NET library for both.

```
var opts = new JsonSerializerOptions
{
    PropertyNamingPolicy = JsonNamingPolicy.CamelCase,
    WriteIndented = true,
    DefaultIgnoreCondition = JsonIgnoreCondition.WhenWritingNull
};
string json = JsonSerializer.Serialize(new Txn("T1", 10m, "OK", "VISA"), opts);
Txn? back = JsonSerializer.Deserialize<Txn>(json, opts);
using JsonDocument doc = JsonDocument.Parse(json);
decimal amt = doc.RootElement.GetProperty("amount").GetDecimal();
```

⚡ Tricky Point

System.Text.Json is **case-sensitive** by default when deserializing (Newtonsoft is not). Set `PropertyNameCaseInsensitive = true` or use `JsonSerializerDefaults.Web` - a common reason for "all properties are null after deserialization".

15.10 Dependency Injection

✎ Definition

Dependency Injection means a class **receives its dependencies** (usually through its constructor) instead of creating them. A DI container wires them up with Transient, Scoped or Singleton lifetimes.

DI means a class receives its dependencies (usually interfaces) from outside instead of creating them with `new`. It enables loose coupling and testing.



Transient = new each time • Scoped = one per request • Singleton = one for app lifetime

Figure 15.1 - Registering and injecting services

```
// Program.cs (ASP.NET Core / Worker Service)
builder.Services.AddSingleton<IClock, SystemClock>();
builder.Services.AddScoped<ITxnRepository, SqlTxnRepository>();
builder.Services.AddTransient<IFeeCalculator, FeeCalculator>();
builder.Services.AddKeyedScoped<IGateway, VisaGateway>("visa");      // .NET 8 keyed services

public class ReconService(ITxnRepository repo, ILogger<ReconService> log)   // constructor injection
{
    public async Task RunAsync() { var t = await repo.GetPendingAsync(); log.LogInformation("{n}", t.Count); }
}
```

Lifetime	Instance	Typical use
Transient	New every time it's requested	Lightweight, stateless services
Scoped	One per scope (HTTP request)	DbContext, repositories, unit of work
Singleton	One for the whole app	Caches, configuration, HttpClient factories

⚡ Tricky Point

Captive dependency: injecting a Scoped service (like `DbContext`) into a Singleton makes the scoped instance live forever - stale data and thread-safety bugs. In a Worker Service / BackgroundService (which is a singleton), create a scope with `IServiceScopeFactory.CreateScope()` for each unit of work.

15.11 Common Design Patterns in C#

✎ Definition

Design patterns are reusable solutions to common design problems, for example Singleton, Factory, Builder, Strategy, Observer, Decorator, Repository and Adapter.

Pattern	Purpose	C# example
Singleton	One instance	`Lazy<T>` or DI `AddSingleton`
Factory / Factory Method	Create objects without exposing concrete classes	`IGatewayFactory.Create("visa")`
Builder	Build complex objects step by step	`StringBuilder`, `WebApplication.CreateBuilder`
Strategy	Swap algorithms at runtime	`IFeeStrategy` implementations per card network
Repository + Unit of Work	Abstract data access	EF Core `DbSet` + `SaveChanges`
Decorator	Add behaviour by wrapping	Caching repository wrapping SQL repository

Pattern	Purpose	C# example
Observer	Notify subscribers	C# events, `IObservable<T>`
Adapter	Make incompatible interfaces work together	Wrap a legacy SOAP client behind `IPaymentGateway`
Template Method	Fixed algorithm skeleton with overridable steps	Abstract `FileProcessor.Process()` calling `Parse()`
Mediator / CQRS	Decouple request senders and handlers	MediatR `Send(new GetOrderQuery(id))`

```
// Strategy pattern example
public interface IFeeStrategy { decimal Fee(decimal amt); }
public class VisaFee : IFeeStrategy { public decimal Fee(decimal a) => a * 0.018m; }
public class RupayFee : IFeeStrategy { public decimal Fee(decimal a) => 0m; }

public class Checkout(IFeeStrategy strategy)
{
    public decimal Total(decimal amt) => amt + strategy.Fee(amt);
}
var total = new Checkout(new VisaFee()).Total(1000);   // 1018.000
```

15.12 Preprocessor Directives and Conditional Compilation

✎ Definition

Preprocessor directives (`#if`, `#define`, `#region`, `#pragma`...) are instructions processed before compilation, used to include or exclude code by build symbol such as DEBUG.

```
#if DEBUG
    Console.WriteLine("Debug build");
#endif
#region Helpers      // collapsible region in the editor
#endregion
#pragma warning disable CS0618
[Conditional("DEBUG")] void Trace(string m) { }   // calls removed in Release builds
```

15.13 Date, Time and Numeric Formatting

✎ Definition

Working with dates and times (`DateTime`, `DateTimeOffset`, `DateOnly`, `TimeSpan`) and turning values into text with **format strings** (`"N2"`, `"C"`, `"yyyy-MM-dd"`) and cultures.

```
DateTime now = DateTime.Now;          // local time
DateTime utc = DateTime.UtcNow;       // store this in databases
DateTimeOffset dto = DateTimeOffset.Now;   // includes offset - safest for APIs
DateOnly d = DateOnly.FromDateTime(now);   // .NET 6
TimeOnly t = TimeOnly.FromDateTime(now);
TimeSpan diff = DateTime.Today - new DateTime(2025, 1, 1);
string s1 = now.ToString("yyyy-MM-dd HH:mm:ss");   // MM = month, mm = minutes!
DateTime parsed = DateTime.ParseExact("20250131", "yyyyMMdd", CultureInfo.InvariantCulture);
string money = 1234567.891m.ToString("N2");          // 1,234,567.89
string cur = 1500m.ToString("C", new CultureInfo("en-IN"));   // currency with Indian grouping
string pct = 0.256.ToString("P1");                   // 25.6 %
string pad = 42.ToString("D6");                      // 000042
```

> ⚡ **Tricky Point**
>
> `MM` is month, `mm` is minutes; `HH` is 24-hour, `hh` 12-hour. `DateTime.Now` in a server app depends on the server's time zone - use UTC for storage and comparisons. `DateTime` is a struct: `date.AddDays(1)` returns a **new** value (forgetting to assign it is a common bug).

16. C# Version History and Modern Features

16.1 Feature Timeline

✎ Definition

A version-by-version list of the major language features C# has added, from C# 1.0 (2002) to C# 14 (.NET 10).

Version (year, .NET)	Headline features
C# 1.0 (2002, .NET Fx 1.0)	Classes, structs, interfaces, events, properties, delegates
C# 2.0 (2005)	Generics, nullable types, iterators (`yield`), anonymous methods, partial classes, static classes
C# 3.0 (2007)	LINQ, lambdas, `var`, extension methods, anonymous types, object/collection initialisers, auto-properties, expression trees
C# 4.0 (2010)	`dynamic`, named & optional parameters, generic covariance/contravariance
C# 5.0 (2012)	`async` / `await`, caller info attributes, foreach variable capture fix
C# 6.0 (2015)	String interpolation `$""`, `nameof`, `?.`, expression-bodied members, auto-property initialisers, `using static`, exception filters
C# 7.0 - 7.3 (2017-18)	Tuples & deconstruction, pattern matching (`is`, switch), `out var`, local functions, `ref` returns, discards, `default` literal, `in` params, `Span<T>`, `readonly struct`, `private protected`
C# 8.0 (2019, .NET Core 3)	Nullable reference types, switch expressions, property/positional patterns, async streams, ranges & indices, default interface methods, `using` declarations, `??=`, static local functions
C# 9.0 (2020, .NET 5)	Records, init-only setters, top-level statements, `and/or/not` patterns, relational patterns, target-typed `new()`, covariant returns, function pointers, module initialisers
C# 10 (2021, .NET 6)	Global usings, file-scoped namespaces, record structs, extended property patterns, constant interpolated strings, lambda natural types, parameterless struct constructors
C# 11 (2022, .NET 7)	Raw string literals `"""`, list patterns, `required` members, generic math (static abstract interface members), `file` types, UTF-8 literals `"abc"u8`, `>>>`
C# 12 (2023, .NET 8 LTS)	Primary constructors for classes/structs, collection expressions `[1, 2, 3]` and spread `..`, alias any type, default lambda parameters, inline arrays, `ref readonly` params
C# 13 (2024, .NET 9)	`params` collections (Span, List…), new `Lock` type, `\e` escape, implicit index `^` in initialisers, `ref`/`unsafe` in async & iterators, `allows ref struct`, partial properties

Version (year, .NET)	Headline features
C# 14 (2025, .NET 10 LTS)	Extension members (extension properties/blocks), `field` keyword in properties, null-conditional assignment `a?.B = x`, `nameof` with unbound generics, implicit Span conversions, partial constructors/events, lambda parameter modifiers without types

16.2 Modern Features in Action

✎ Definition

Short, practical examples of recent C# features (records, pattern matching, primary constructors, collection expressions and more) used together in real code.

```
// Collection expressions + spread (C# 12)
int[] a = [1, 2]; int[] b = [3, 4];
int[] all = [..a, ..b, 5];                  // 1,2,3,4,5
List<string> empty = [];

// Required members (C# 11)
public class Customer { public required string Id { get; init; } }
var c = new Customer { Id = "C1" };         // omitting Id = compile error

// Raw strings + UTF-8 literals (C# 11)
string sql = """
    SELECT * FROM Txn
    WHERE Status = 'SUCCESS'
    """;
ReadOnlySpan<byte> header = "TXN"u8;

// Alias any type (C# 12)
using Point = (int X, int Y);
Point p = (1, 2);
// Null-conditional assignment (C# 14)
customer?.Address = newAddress;             // assigns only if customer is not null

// field keyword (C# 14)
public string Name { get; set => field = value?.Trim() ?? ""; }
```

16.3 Top-Level Program for a Minimal API (.NET 8+)

✎ Definition

A **Minimal API** is an ASP.NET Core web API written with top-level statements and `app.MapGet/MapPost`, without controllers or a `Startup` class.

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddScoped<ITxnRepository, SqlTxnRepository>();
var app = builder.Build();

app.MapGet("/txns/{id}", async (string id, ITxnRepository repo) =>
    await repo.FindAsync(id) is { } t ? Results.Ok(t) : Results.NotFound());

app.MapPost("/txns", async (Txn t, ITxnRepository repo) =>
{
    await repo.AddAsync(t);
    return Results.Created($"/txns/{t.Id}", t);
});
app.Run();
```

17. Tricky Interview Questions and Output Puzzles

Try to predict each output before reading the answer. These are drawn from the most common C# interview rounds.

17.1 Output Prediction Puzzles

> ✎ **Definition**
>
> Short code snippets where you **predict the exact output**. They test a precise understanding of language rules such as closures, value vs reference types, statics and evaluation order.

Puzzle 1 - String immutability

```
string s = "abc";
s.Replace("a", "x");
Console.WriteLine(s);
```

> ⚡ **Tricky Point**
>
> Answer: **abc**. Strings are immutable; `Replace` returns a new string that was discarded.

Puzzle 2 - Struct copy in a List

```
struct Counter { public int Value; }
var list = new List<Counter> { new Counter() };
var c = list[0]; c.Value = 10;
Console.WriteLine(list[0].Value);
```

> ⚡ **Tricky Point**
>
> Answer: **0**. `list[0]` returns a copy of the struct. (`list[0].Value = 10;` wouldn't even compile.) With an array, `arr[0].Value = 10` works because array indexing gives direct access to the element.

Puzzle 3 - virtual vs new

```
class A { public virtual string Who() => "A"; }
class B : A { public override string Who() => "B"; }
class C : B { public new string Who() => "C"; }
A obj = new C();
Console.WriteLine(obj.Who());
```

> ⚡ **Tricky Point**
>
> Answer: **B**. Virtual dispatch finds the most-derived *override* in the chain. `C.Who` is `new` (hides, not overrides), so the chain stops at B.

Puzzle 4 - Static constructor and field order

```
class Demo
{
    public static int X = Init("X");
    static Demo() => Console.Write("Ctor ");
    static int Init(string n) { Console.Write(n + " "); return 1; }
}
Console.Write("Start ");
var x = Demo.X;
```

⚡ Tricky Point

Answer: **Start X Ctor**. Static field initialisers run just before the static constructor, which runs on first access to the type (here, after "Start" because there's an explicit static ctor).

Puzzle 5 - Deferred execution

```
var list = new List<int> { 1, 2, 3 };
var q = list.Select(x => x * 10);
list.Add(4);
Console.WriteLine(q.Sum());
```

⚡ Tricky Point

Answer: **100**. The query executes at `Sum()`, after 4 was added: 10+20+30+40.

Puzzle 6 - Closure in a for loop

```
var fs = new List<Func<int>>();
for (int i = 0; i < 3; i++) fs.Add(() => i * i);
Console.WriteLine(string.Join(",", fs.Select(f => f())));
```

⚡ Tricky Point

Answer: **9,9,9**. All lambdas capture the same `i`, which is 3 after the loop.

Puzzle 7 - finally and return

```
static int Get()
{
    int x = 10;
    try { return x; }
    finally { x = 20; Console.Write("F "); }
}
Console.WriteLine(Get());
```

⚡ Tricky Point

Answer: **F 10**. The return value is evaluated before `finally` runs; changing `x` later doesn't affect it.

Puzzle 8 - Integer overflow and division

```
int a = int.MaxValue;
Console.WriteLine(a + 1);
Console.WriteLine(7 / 2);
Console.WriteLine(-7 / 2);
Console.WriteLine(-7 % 3);
```

⚡ Tricky Point

Answer: **-2147483648, 3, -3, -1**. Overflow wraps silently (unchecked). Integer division truncates toward zero, and the remainder takes the sign of the dividend.

Puzzle 9 - Equality of objects

```
object a = 5, b = 5;
Console.WriteLine(a == b);
Console.WriteLine(a.Equals(b));
string s1 = "hi", s2 = new string("hi".ToCharArray());
object o1 = s1, o2 = s2;
Console.WriteLine(s1 == s2);
Console.WriteLine(o1 == o2);
```

⚡ Tricky Point

Answer: **False, True, True, False**. `a` and `b` are two separate boxes; `==` on `object` compares references. `Equals` compares values. `s1 == s2` uses the string `==` overload (content), but `o1 == o2` is typed as object, so it compares references.

Puzzle 10 - Nullable arithmetic

```
int? x = null;
int? y = x + 5;
Console.WriteLine(y == null);
Console.WriteLine(x > 0 || x <= 0);
```

⚡ Tricky Point

Answer: **True, False**. Lifted arithmetic with null yields null; any comparison involving null returns false.

Puzzle 11 - Exceptions in async void vs Task

```
async Task ThrowAsync() { await Task.Yield(); throw new InvalidOperationException("boom"); }
var t = ThrowAsync();           // no exception here
Console.WriteLine("After call");
try { await t; } catch (InvalidOperationException e) { Console.WriteLine(e.Message); }
```

⚡ Tricky Point

Answer: **After call** then **boom**. The exception is stored inside the Task and only surfaces when awaited.

Puzzle 12 - Dictionary with mutable key

```
var key = new List<int> { 1 };
var set = new HashSet<List<int>> { key };
key.Add(2);
Console.WriteLine(set.Contains(key));
```

> ⚡ **Tricky Point**
>
> Answer: **True** - surprisingly. `List<T>` uses the default (reference-based) `GetHashCode`, which doesn't change when contents change. With a type whose hash depends on mutable fields (e.g. a record), the answer would be **False**.

Puzzle 13 - Records equality

```
record P(string Name, List<int> Scores);
var l = new List<int> { 1 };
Console.WriteLine(new P("A", l) == new P("A", l));
Console.WriteLine(new P("A", new() { 1 }) == new P("A", new() { 1 }));
```

> ⚡ **Tricky Point**
>
> Answer: **True, False**. Record equality compares members with their own `Equals`; the lists are compared by reference.

Puzzle 14 - Short-circuit and side effects

```
int i = 0;
bool r = (i++ > 0) && (i++ > 0);
Console.WriteLine($"{r} {i}");
bool r2 = (i++ > 5) & (i++ > 0);
Console.WriteLine($"{r2} {i}");
```

> ⚡ **Tricky Point**
>
> Answer: **False 1** then **False 3**. `&&` skips the right side when the left is false; `&` always evaluates both.

17.2 Frequently Asked Conceptual Questions

✎ Definition

The theory questions interviewers ask most often, each with a concise answer you can give in one or two sentences.

Question	Crisp answer
Value type vs reference type?	Value types hold data and copy on assignment; reference types hold a reference to a heap object and copy the reference.
class vs struct vs record?	class = reference & identity; struct = small value type; record = data type with value equality and `with` copying.
const vs readonly vs static readonly?	const = compile-time, baked into callers; readonly = set once at runtime per instance; static readonly = once per type.
ref vs out vs in?	ref = in/out, must be initialised; out = output only, must be assigned in the method; in = read-only by reference.
Abstract class vs interface?	Abstract class shares state + code, single inheritance; interface = contract, multiple implementation.
Overloading vs overriding?	Overloading = same name, different params, compile-time; overriding = virtual method replaced in derived class, runtime.

Question	Crisp answer
override vs new?	override participates in virtual dispatch; new hides and is resolved by reference type.
Boxing?	Converting a value type to object/interface: heap allocation + copy. Unboxing = explicit cast back to the exact type.
IEnumerable vs IQueryable?	IEnumerable executes in memory with delegates; IQueryable builds an expression tree executed by a provider (SQL).
IEnumerable vs ICollection vs IList?	Iterate only; + Count/Add/Remove; + index access.
Dictionary vs Hashtable?	Dictionary is generic/type-safe, throws on missing key; Hashtable stores object, returns null, legacy.
List vs Array?	Array fixed size; List resizes automatically (wraps an array).
Task vs Thread?	Thread = OS thread; Task = higher-level unit of work, usually on the ThreadPool, composable with async/await.
async/await internals?	Compiler creates a state machine; at an incomplete await the method returns a Task and resumes later via continuation.
Dispose vs Finalize?	Dispose = deterministic cleanup called by code (`using`); Finalize = GC-called, non-deterministic, for unmanaged resources.
What is GC? Generations?	Automatic memory reclamation of unreachable objects; Gen 0/1/2 + LOH based on object age.
Delegate vs event?	Event wraps a delegate so outsiders can only subscribe/unsubscribe, not invoke or reset it.
Func vs Action vs Predicate?	Func returns a value; Action returns void; Predicate<T> returns bool.
Why override GetHashCode with Equals?	Hash-based collections use the hash first; equal objects must have equal hashes.
What is a sealed class?	Cannot be inherited; enables JIT optimisations (devirtualisation).
Can a static class have a constructor?	Yes - only a static constructor.
Can an interface have fields?	No instance fields; static fields are allowed since C# 8.
Can we have a private constructor?	Yes - to prevent external instantiation (singleton, static factory).
What is a deadlock with .Result?	The blocked thread holds the context the continuation needs; fix with async all the way / ConfigureAwait(false).
Shallow vs deep copy?	Shallow copies references (shared nested objects); deep copies nested objects too.
Is string a value type?	No, a reference type that is immutable and overloads == for value semantics.
What is the use of `using`?	Imports namespaces; and as a statement/declaration it guarantees Dispose via try/finally.
What is `yield`?	Creates a lazy iterator state machine producing items on demand.

Question	Crisp answer
What does `sealed override` do?	Overrides a method and prevents further overriding in derived classes.

17.3 Coding Questions to Practise

✎ Definition

Common hands-on coding problems (strings, arrays, collections, LINQ) with sample solutions, for practising clean, correct C# under interview conditions.

```
// 1) Reverse a string without built-ins
string Rev(string s)
{
    var c = s.ToCharArray();
    for (int i = 0, j = c.Length - 1; i < j; i++, j--) (c[i], c[j]) = (c[j], c[i]);
    return new string(c);
}

// 2) Palindrome check
bool IsPal(string s) => s.SequenceEqual(s.Reverse());
// 3) First non-repeating character
char? FirstUnique(string s) => s.GroupBy(c => c).FirstOrDefault(g => g.Count() == 1)?.Key;

// 4) Two Sum with Dictionary - O(n)
int[] TwoSum(int[] nums, int target)
{
    var seen = new Dictionary<int, int>();
    for (int i = 0; i < nums.Length; i++)
    {
        if (seen.TryGetValue(target - nums[i], out int j)) return new[] { j, i };
        seen[nums[i]] = i;
    }
    return Array.Empty<int>();
}

// 5) Second highest salary
var second = employees.Select(e => e.Salary).Distinct()
                      .OrderByDescending(s => s).Skip(1).FirstOrDefault();
// 6) Word frequency
var freq = text.Split(' ', StringSplitOptions.RemoveEmptyEntries)
               .GroupBy(w => w.ToLower()).ToDictionary(g => g.Key, g => g.Count());

// 7) Find duplicates in an array
var dups = arr.GroupBy(x => x).Where(g => g.Count() > 1).Select(g => g.Key);

// 8) Thread-safe singleton
public sealed class Singleton
{
    private static readonly Lazy<Singleton> _i = new(() => new Singleton());
    public static Singleton Instance => _i.Value;
    private Singleton() { }
}

// 9) Fibonacci with memoisation
long Fib(int n, Dictionary<int, long>? memo = null)
{
    memo ??= new();
    if (n < 2) return n;
    if (memo.TryGetValue(n, out var v)) return v;
    return memo[n] = Fib(n - 1, memo) + Fib(n - 2, memo);
}

// 10) Run 100 HTTP calls with max 10 concurrent
using var gate = new SemaphoreSlim(10);
var tasks = urls.Select(async u =>
{
    await gate.WaitAsync();
    try { return await http.GetStringAsync(u); }
```

```
    finally { gate.Release(); }
});
var pages = await Task.WhenAll(tasks);
```

18. Quick Revision Cheat Sheet

18.1 Syntax at a Glance

✎ Definition

A compact reference of everyday C# syntax: declarations, control flow, classes, collections, LINQ and async, in one place for quick lookup.

```
// Types & variables
int n = 5; var s = "x"; const int Max = 10; int? maybe = null; decimal m = 9.99m;
// Strings
$"{name} {amt:N2}"   @"C:\path"   """raw"""   sb.Append(x)
// Null handling
a?.B   a ?? b   a ??= b   a!   x is null   x is not null
// Conditions
cond ? x : y      value switch { 1 => "one", > 5 => "big", _ => "other" }
// Loops
for (int i = 0; i < n; i++)   foreach (var x in xs)   while (c)   do { } while (c);
// Methods
int Add(int a, int b = 0) => a + b;   void M(ref int x, out int y, in Big z, params int[] r)
// Classes
public class C : Base, IFace { public int P { get; init; } public C(int p) : base() => P = p; }
public record R(string A, int B);   var r2 = r with { B = 2 };
// Generics
T Max<T>(T a, T b) where T : IComparable<T> => a.CompareTo(b) > 0 ? a : b;
// Delegates & lambdas
Func<int,int> sq = x => x * x;   Action<string> log = Console.WriteLine;   event EventHandler? E;
// LINQ
xs.Where(p).Select(f).OrderBy(k).GroupBy(k).ToList()   xs.Any()  xs.First()  xs.Sum(f)
// Async
async Task<T> GetAsync(CancellationToken ct) { var r = await Call(ct); return r; }
await Task.WhenAll(t1, t2);
// Exceptions
try { } catch (Ex e) when (cond) { throw; } finally { }
// Resources
using var conn = new SqlConnection(cs);
```

18.2 Top 25 Tricky Points (Revise Before Interviews)

✎ Definition

The 25 most important "gotchas" from this guide, gathered for fast revision before an interview.

- `0.1 + 0.2 != 0.3` with double - use `decimal` for money.
- Integer overflow wraps silently unless `checked`.
- `Math.Round(2.5)` = 2 (banker's rounding).
- `(int)2.9` = 2 (cast truncates); `Convert.ToInt32(2.5)` = 2, `Convert.ToInt32(3.5)` = 4.
- `Convert.ToInt32(null)` = 0 but `int.Parse(null)` throws.
- `const` values are baked into calling assemblies.
- Everything is passed **by value** by default - including references.
- Strings are immutable: `s.Replace(...)` without assignment does nothing.
- `==` on `object` compares references even for boxed equal numbers.
- Override `GetHashCode` whenever you override `Equals`.
- `new` hides, `override` overrides - the reference type decides for `new`.

- Never call virtual methods from constructors.
- Struct in `List<T>` - indexer returns a copy.
- `List.Sort` is unstable; LINQ `OrderBy` is stable.
- `Dictionary[key]` throws for a missing key; `Hashtable[key]` returns null. Use `TryGetValue`.
- Don't modify a collection inside `foreach` over it.
- `SortedSet` drops items whose comparer returns 0.
- LINQ is deferred - multiple enumeration re-runs the query.

- `SingleOrDefault` throws when there are multiple matches.
- `All()` on an empty sequence returns true; `Max()` on empty throws.
- `for` loop closures capture one shared variable; `foreach` gets a fresh one (C# 5+).
- `async void` exceptions crash the process; use `async Task`.
- `.Result`/`.Wait()` can deadlock; go async all the way.
- No `await` inside `lock` - use `SemaphoreSlim`.
- Scoped services inside singletons = captive dependency bug.

18.3 Suggested Study Roadmap

✎ Definition

A suggested order of study, from fundamentals to advanced topics, with milestones that build practical C# skill step by step.

Week	Focus	Practice
1	Chapters 1-4: runtime, types, control flow, methods	Write 20 small console programs; predict outputs
2	Chapters 5-9: strings, memory, OOP, records, exceptions	Model a Bank/Payment domain with classes, interfaces, custom exceptions
3	Chapters 10-13: collections, generics, delegates, LINQ	Re-implement a reconciliation: match two lists of transactions with Dictionary and LINQ
4	Chapters 14-16: async, threading, DI, patterns, modern C#	Build a minimal API + background worker with cancellation and SemaphoreSlim throttling
5	Chapter 17: puzzles & coding questions	Explain each answer aloud in one minute; time yourself on coding questions

i Note

Keep this guide next to your IDE. When something surprises you while coding, find the matching "Tricky Point" box - most real bugs come from exactly those behaviours.